

Vidya Academy of Science & Technology

(A unit of Vidya International Charitable Trust)

Approved by AICTE & Affiliated to APJ Abdul Kalam Technological University

Accredited by NAAC with "B++" grade



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PCCSL 507: NETWORKS LAB

LAB MANUAL

PREPARED BY:

Ms. JUCY VAREED

APPROVED BY:

HOD, CSE

Vision of the Institution

To be a premier institution in engineering education and research, nurturing innovative professionals for sustainable societal development.

Mission of the Institution

- M1. Provide quality engineering education that empowers students with strong technical competence and analytical problem solving abilities.
- M2. Promote research, innovation, entrepreneurship and industry collaboration to develop sustainable technological solutions.
- M3. Develop ethical and socially responsible professionals with leadership qualities, lifelong learning abilities and a commitment to societal well-being.

Vision of the Department

To excel in Computer Science and Engineering education and research, nurturing innovative and ethically responsible professionals for sustainable technological development.

Mission of the Department

- M1. Impart quality education in Computer Science and Engineering by providing strong theoretical foundations, practical skills and exposure to emerging technologies.
- M2. Promote research, innovation, entrepreneurship and industry collaboration to develop sustainable and socially relevant computing solutions.
- M3. Develop graduates with professional competence, ethical values, logical and analytical thinking skills, leadership qualities and a commitment to lifelong learning and societal well-being.

Program Educational Objectives (PEOs)

PEO1. Graduates will be able to apply strong foundations in Computer Science and Engineering to design and develop software solutions using modern tools and technologies.

PEO2. Graduates will engage in research, innovation and entrepreneurship to develop sustainable and industry-relevant computing solutions.

PEO3. Graduates will demonstrate professional ethics, leadership and effective teamwork while contributing responsibly to society through their engineering practice.

PEO4. Graduates will pursue continuous professional development and lifelong learning to remain proficient in evolving technologies throughout their careers.

Program Specific Outcomes (PSOs)

PSO1 Build robust software systems and intelligent applications using modern programming languages, frameworks and algorithms.

PSO2 Apply core concepts in networking and cloud computing to design efficient and secure systems for real-world applications

PSO3 Develop innovative, sustainable and socially relevant software solutions through interdisciplinary collaboration and industry engagement.

Program Outcomes (Pos)

Computer Science & Engineering graduates will be able to:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering

problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Sustainable Development Goals (SDGs)

SDG 1 - No Poverty: End poverty in all its forms everywhere.

SDG 2 - Zero Hunger: End hunger, achieve food security and improved nutrition and promote sustainable agriculture.

SDG 3 - Good Health and Well-being: Ensure healthy lives and promote well-being for all at all ages.

SDG 4 - Quality Education: Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all.

SDG 5 - Gender Equality: Achieve gender equality and empower all women and girls.

SDG 6 - Clean Water and Sanitation: Ensure availability and sustainable management of water and sanitation for all.

SDG 7 - Affordable and Clean Energy: Ensure access to affordable, reliable, sustainable and modern energy for all.

SDG 8 - Decent Work and Economic Growth: Promote sustained, inclusive and sustainable economic growth, full and productive employment and decent work for all.

SDG 9 - Industry, Innovation and Infrastructure: Build resilient infrastructure, promote inclusive and sustainable industrialization and foster innovation.

SDG 10 - Reduced Inequalities: Reduce inequality within and among countries.

SDG 11 - Sustainable Cities and Communities: Make cities and human settlements inclusive, safe, resilient and sustainable.

SDG 12 - Responsible Consumption and Production: Ensure sustainable consumption and production patterns.

SDG 13 - Climate Action : Take urgent action to combat climate change and its impacts.

SDG 14 - Life Below Water : Conserve and sustainably use the oceans, seas and marine resources for sustainable development.

SDG 15 -Life on Land: Protect, restore and promote sustainable use of terrestrial ecosystems, sustainably manage forests, combat desertification, and halt and reverse land degradation and halt biodiversity loss.

SDG 16 - Peace, Justice and Strong Institutions: Promote peaceful and inclusive societies for sustainable development, provide access to justice for all and build effective, accountable and inclusive institutions at all levels.

SDG 17 -Partnerships for the Goals: Strengthen the means of implementation and revitalize the global partnership for sustainable development.

PCCSL 507 NETWORKS LAB

(Common to CS/CD/CM/CB/CU/CI)

Course Code	PCCSL507	CIE Marks	50
Teaching Hours/Week (L:T:P:R)	0:0:3:0	ESE Marks	50
Credits	2	Exam Hours	2 Hrs. 30 Min.
Prerequisites (if any)	None	Course Type	Lab

Course Objectives:

1. To provide hands-on experience in network programming using Linux System calls and network monitoring tools.
2. To comprehend the implementation of network protocols and algorithms, and configuration of network layer services using network simulators.

List of Experiments

Expt. No.	Experiments
	Warm up
1	Familiarize Linux networking commands - ifconfig, ifplugstatus, iftop, ping, ip, traceroute, mtr, netstat, whois, nmap, nmcli, speedtest-cli, bmon, nslookup, tcpdump
	Wireshark based
2	Start your web browser and clear the browser's cache memory. Open Wireshark and start capturing. Then visit any webpage of your choice. Type http in the filter field of the Wireshark and click Apply so that only HTTP messages are displayed. After enough packets have been captured, select Capture from the pull-down menu and select Stop to stop capturing. Determine source/destination IP, medium format/language/encoding/charset accepted, URL and user agent, response IP addresses, status code, last-modified time, content-length, response time and HTTP version.
3	Compose an e-mail and address it to yourself, but do not send it yet. Capture the

	SMTP exchange in Wireshark and answer questions on IP addresses, port numbers, SMTP commands/response codes and the encapsulated IMF packet.
4	Clear the DNS cache and browser cache. Capture DNS query/response packets for your college website and answer questions on packet numbers, transport protocol, ports, transaction ID, flag field, QR bit and resource record counts.
	Socket programming based
5	Client-Server communication using TCP:- The client inputs an integer N and creates a square matrix of order N by populating the matrix with random numbers in the range [1,50]. It then sends the matrix to the server which identifies the matrix type (upper triangular, lower triangular, diagonal). The server then informs the type (as a string) to the client which it prints.
6	Client-Server communication using UDP:- Design a client-server application that translates new-generation English abbreviations (tbh, ig, tbf, atm, irl, lol, asap, omg, ttyl, idk, nvm) to formal English.
7	Implement a multi-user chat server using TCP as the transport layer protocol.
8	Implement a concurrent Time Server application using UDP to execute the program at a remote server. The client sends a time request to the server which sends its system time back. The client then displays the received time value.
9	Develop a concurrent file server that will provide the file requested by the client if it exists. If not, the server sends an appropriate message to the client along with its process ID (PID).
10	Develop a packet-capturing application using raw sockets.
	Cisco's Packet Tracer based
11	Familiarizing router commands - mode switching, router information, routing protocol status, routing table, saving configuration, command history, router clock, list of hosts, interface statistics, controller type, configuring serial and ethernet interfaces.
12	Set up static routing for a sample network. Display the routing table and verify connectivity using ping.
13	Implement RIPv2 routing for the sample network. Display the routing table and verify connectivity using ping.
14	Implement OSPF routing for the sample network. Display the routing table and verify connectivity using ping.
15	As network administrator, configure a campus network so that only Host_B can communicate with a given subnet; verify with ping tests.
16	Given a college network address with 20 subnets, block access to a specific

	hackathon website hosted on a server without denying its other services.
17	Interconnect the subnets of an IPv6-based network using RIPng.

Course Outcomes (COs)

At the end of the course, students should be able to:

CO	Course Outcome	Bloom's KL
CO1	Understand the working of application layer protocols by analyzing the pertinent headers in actual data packets captured using network monitoring tools.	K3
CO2	Exploit the client server paradigm to develop real time networking applications using transport layer protocols.	K3
CO3	Employ IPv4 and IPv6 addressing, subnetting to efficiently design networks.	K3
CO4	Simulate core networking concepts using a network simulator.	K3

Note: K1-Remember, K2-Understand, K3-Apply, K4-Analyse, K5-Evaluate, K6-Create

CO - PO/PSO MAPPING:

CO Number	PO STRENGTH												PSO STRENGTH	
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	3	3	3	3	3						3		2	
CO2	3	3	3	3		2					3		2	2
CO3	3	3	3	3							3		2	2
CO4	3	3	3	3	3						3		2	

GENERAL INSTRUCTIONS(For the Lab)Preparation

Understand the objective and networking concept of the experiment.

Study the required socket functions and system calls in C.

Review sample client–server programs before implementation.

Ensure familiarity with TCP/UDP communication models.

Lab Setup

Use a system with a C compiler (gcc) and Linux/UNIX environment.

Ensure necessary header files for socket programming are available.

Arrange terminals or systems for client and server execution.

Check network connectivity and firewall permissions if required.

Documentation

Write clear comments explaining logic and function usage.

Maintain a lab record with aim, algorithm, program code, and output.

Record observations and errors encountered during execution.

Include screenshots or terminal outputs where required.

Coding Standard

Follow proper indentation and readable formatting.

Use meaningful variable and function names.

Handle errors for all system calls.

Avoid hard-coding values; use constants where possible.

Keep client and server code modular and well-structured.

Testing & Debugging

Compile code without warnings using gcc.

```
gcc -o server server.c
```

```
gcc -o client client.c
```

Run the server program on the remote server: ./server

Run the client program on the same or another machine: `./client`

Test programs with valid and invalid inputs.

Verify proper data transmission between client and server.

Use debugging tools (`printf`, `gdb`) to trace errors.

Ensure all sockets are closed properly after execution.

INDEX

Sl No	Experiment Name	Page No
1	Familiarize Linux networking commands - ifconfig, ifplugstatus, iftop, ping, ip, traceroute, mtr, netstat, whois, nmap, nmcli, speedtest-cli, bmon, nslookup, tcpdump	13
	Wireshark based	
2	Start your web browser and clear the browser's cache memory. Open Wireshark and start capturing. Then visit any webpage of your choice. Type http in the filter field of the Wireshark and click Apply so that only HTTP messages are displayed. After enough packets have been captured, select Capture from the pull-down menu and select Stop to stop capturing. Determine source/destination IP, medium format/language/encoding/charset accepted, URL and user agent, response IP addresses, status code, last-modified time, content-length, response time and HTTP version.	18
3	Compose an e-mail and address it to yourself, but do not send it yet. Capture the SMTP exchange in Wireshark and answer questions on IP addresses, port numbers, SMTP commands/response codes and the encapsulated IMF packet.	21
4	Clear the DNS cache and browser cache. Capture DNS query/response packets for your college website and answer questions on packet numbers, transport protocol, ports, transaction ID, flag field, QR bit and resource record counts.	25
	Socket programming based	
5	Client-Server communication using TCP:- The client inputs an integer N and creates a square matrix of order N by populating the matrix with random numbers in the range [1,50]. It then sends the matrix to the server which identifies the matrix type (upper triangular, lower triangular, diagonal). The server then informs the type (as a string) to the client which it prints.	28
6	Client-Server communication using UDP:- Design a client-server application that translates new-generation English abbreviations (tbh, ig, tbf, atm, irl, lol, asap, omg, ttyl, idk, nvm) to formal English.	34
7	Implement a multi-user chat server using TCP as the transport layer protocol.	41

8	Implement a concurrent Time Server application using UDP to execute the program at a remote server. The client sends a time request to the server which sends its system time back. The client then displays the received time value.	47
9	Develop a concurrent file server that will provide the file requested by the client if it exists. If not, the server sends an appropriate message to the client along with its process ID (PID).	52
10	Develop a packet-capturing application using raw sockets.	59
	Cisco's Packet Tracer based	
11	Familiarizing router commands - mode switching, router information, routing protocol status, routing table, saving configuration, command history, router clock, list of hosts, interface statistics, controller type, configuring serial and ethernet interfaces.	64
12	Set up static routing for a sample network. Display the routing table and verify connectivity using ping.	71
13	Implement RIPv2 routing for the sample network. Display the routing table and verify connectivity using ping.	76
14	Implement OSPF routing for the sample network. Display the routing table and verify connectivity using ping.	82
15	As network administrator, configure a campus network so that only Host_B can communicate with a given subnet; verify with ping tests.	87
16	Given a college network address with 20 subnets, block access to a specific hackathon website hosted on a server without denying its other services.	92
17	Interconnect the subnets of an IPv6-based network using RIPng.	97

EXP NO: 1**FAMILIARIZE LINUX NETWORKING COMMANDS****AIM**

To study and demonstrate the usage of essential Linux networking commands - ifconfig, ifplugstatus, iftop, ping, ip, traceroute, mtr, netstat, whois, nmap, nmcli, speedtest-cli, bmon, nslookup and tcpdump - for network interface configuration, connectivity testing, monitoring and troubleshooting.

LEARNING OUTCOME

- Explain the purpose of Linux networking commands and their role in system administration.
- Use ip to view and configure network interfaces and IP addresses.
- Verify network connectivity using ping.
- Display and analyze routing information using ip route.
- Identify network configuration details with commands such as hostname, ip addr, and ip link.
- Check active network connections and listening ports using ss or netstat.
- Retrieve DNS information using tools like nslookup, dig, or host.
- Trace network paths using traceroute or tracepath.
- Test remote connectivity and open ports using telnet or nc (Netcat).
- Troubleshoot basic network issues by combining multiple Linux networking commands.

DESCRIPTION

These commands together form a complete toolkit for Linux network administration. ifconfig, ip and nmcli are used to view and configure network interfaces; ping, traceroute and mtr test reachability and trace the path to a destination; iftop, netstat and bmon monitor bandwidth usage, active connections and interface statistics; nslookup and whois resolve domain names and query registration information; nmap performs host and port discovery for security auditing; and tcpdump captures raw packets for protocol-level analysis.

PROCEDURE

1. Log in to the Linux terminal on the lab machine.
2. Execute each of the fifteen networking commands listed below, one at a time, using the syntax and options shown.
3. Observe and record the output produced by each command.

4. Interpret the fields displayed (IP address, MAC address, RTT, hop count, port state, DNS records, etc.).

1. ifconfig

A classic Linux command used to view and configure network interfaces. It has been deprecated in favour of the ip command but is still widely used. Its output includes the interface name, IP address, MAC address and network statistics (packets sent/received).

```
student@cse-hwl-030:~$ ifconfig
enp4s0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 10.11.1.135 netmask 255.0.0.0 broadcast 10.255.255.255
inet6 fe80::343b:6a03:2841:bab4 prefixlen 64 scopeid 0x20<link>
ether 74:56:3c:58:7e:1f txqueuelen 1000 (Ethernet)
RX packets 532008 bytes 296326338 (296.3 MB)
RX errors 0 dropped 5436 overruns 0 frame 0
TX packets 115565 bytes 42990464 (42.9 MB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
inet6 ::1 prefixlen 128 scopeid 0x10<host>
```

2. ifplugstatus

Checks all network interfaces and reports their link (cable) status.

```
$ ifplugstatus eth0
eth0: link beat detected
```

Exit codes commonly used:

- 0 -> link detected
- 1 -> unplugged / no link
- >1 -> error (interface not found, permissions issue, etc.)

3. iftop

A real-time network bandwidth monitoring tool, similar to top but for network traffic instead of CPU/memory.

Syntax: iftop [options]

- i interface Monitor a specific network interface (e.g., eth0)
- n Do not resolve hostnames; show IP addresses only
- N Display port numbers instead of service names

- P Show both port numbers and host information
- B Display bandwidth in bytes instead of bits
- h Display help information

4. ping

Tests connectivity between the local host and a remote host by sending ICMP Echo Request packets and waiting for Echo Reply packets.

```
student@cse-hwl-030:~$ ping www.google.com
```

```
PING www.google.com (142.251.155.119) 56(84) bytes of data.
```

```
64 bytes from 142.251.155.119: icmp_seq=1 ttl=119 time=28.1 ms
```

```
64 bytes from 142.251.155.119: icmp_seq=2 ttl=119 time=31.7 ms
```

```
64 bytes from 142.251.155.119: icmp_seq=3 ttl=119 time=30.9 ms
```

```
^C
```

```
--- www.google.com ping statistics ---
```

```
6 packets transmitted, 6 received, 0% packet loss, time 5004ms
```

```
rtt min/avg/max/mdev = 27.938/29.581/31.743/1.404 ms
```

5. ip

A modern replacement for ifconfig and route, used to view and configure network interfaces, IP addresses and routes.

```
student@cse-hwl-030:~$ ip addr
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
```

```
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

```
inet 127.0.0.1/8 scope host lo
```

```
2: enp4s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
```

```
link/ether 74:56:3c:58:7e:1f brd ff:ff:ff:ff:ff:ff
```

```
inet 10.11.1.135/8 brd 10.255.255.255 scope global noprefixroute enp4s0
```

6. Traceroute

7. Shows the number of hops taken to reach a destination and the path packets travel through the network.

```
$ traceroute to 4.2.2.2 (4.2.2.2), 30 hops max, 60 byte packets
```

```
1 192.168.50.1 (192.168.50.1) 0.217 ms 0.624 ms 0.133 ms
```

```
2 227.18.106.27.mysipl.com (27.106.18.227) 2.343 ms 1.910 ms 1.799 ms
```

```
3 221-231-119-111.mysipl.com (111.119.231.221) 4.334 ms 4.001 ms 5.619 ms
```

```
4 10.0.0.5 (10.0.0.5) 5.386 ms 6.490 ms 6.224 ms
```

```
5 gi0-0-0.dgw1.bom2.pacific.net.in (203.123.129.25) 7.798 ms 7.614 ms 7.378 ms
```

7. mtr

"My Traceroute" combines the features of ping and traceroute into a single, continuously updating view of latency and packet loss per hop.

```
$ mtr google.com
```

```
Host Loss% Snt Last Avg Best Wrst
```

```
1. 192.168.1.1 0.0% 10 1.2 1.1 0.9 1.5
```

```
2. 10.0.0.1 0.0% 10 5.3 5.1 4.8 6.0
```

```
3. 142.250.190.78 0.0% 10 15.8 15.2 14.9 16.5
```

8. netstat

Displays active network connections, routing tables, interface statistics and listening ports.

Syntax: netstat [options]

-t TCP connections -u UDP connections -l Listening ports only

-n Numeric addresses -r Routing table -i Interface statistics

-a All connections

```
student@cse-hwl-030:~$ netstat
```

```
Proto Recv-Q Send-Q Local Address Foreign Address State
```

Non-authoritative answer:

Name: google.com

Address: 142.250.77.142

Name: google.com

Address: 2404:6800:4007:80f::200e

15. tcpdump

Captures and analyses network packets in real time; widely used for troubleshooting, security analysis and protocol debugging.

Capture on an interface : tcpdump -i eth0

Readable output : tcpdump -n -i eth0

Only HTTP traffic (port 80) : tcpdump -i eth0 port 80

Verbose detail : tcpdump -vvv -i eth0

Save capture to a file : tcpdump -i eth0 -w capture.pcap

Read a saved capture : tcpdump -r capture.pcap

MODEL VIVA QUESTIONS:

1. What is the difference between ifconfig and the ip command?
2. What protocol does the ping command use internally?
3. How does traceroute determine the path to a destination?
4. What is the difference between mtr and traceroute?
5. What information does nmap provide during a port scan?
6. What is the purpose of the nslookup command?

RESULT

The Linux networking commands were executed successfully, and network configuration, connectivity, traffic and performance information were observed and analysed.

EXP NO: 2**WIRESHARK-BASED ANALYSIS OF HTTP****AIM**

Start a web browser and clear its cache. Open Wireshark, start capturing, and visit any webpage of choice. Apply the display filter http so that only HTTP messages are shown, and stop the capture once sufficient packets are collected. Using the captured information, determine:

- The source and destination IP addresses of the first GET message.
- The medium format, language, encoding and character set the client can accept (from the first GET message).
- The URL of the website and the User-Agent (from the first GET message).
- The source and destination IP addresses of the first response message.
- The status code of the first response message.
- When the retrieved HTML file was last modified at the server.
- The value of the Content-Length field of the first response message.
- The time elapsed between the GET request and its corresponding response.
- The HTTP version used by the browser.

LEARNING OUTCOME

By the end of this experiment, learners will be able to:

1. Understand the role of Wireshark as a network protocol analyzer.
2. Capture and filter HTTP packets using Wireshark.
3. Identify and distinguish between HTTP request and response messages.
4. Interpret HTTP methods (GET, POST, etc.), status codes, and header fields.
5. Analyze the sequence of HTTP communication between a client and a web server.
6. Examine packet details to understand application-layer data exchange.
7. Apply packet filtering techniques to isolate HTTP traffic for analysis.
8. Correlate captured network traffic with the HTTP protocol workflow.
9. Develop basic network troubleshooting skills using packet capture and analysis.
10. Gain practical experience in monitoring and analyzing application-layer network protocols

DESCRIPTION

This experiment introduces students to the use of Wire shark, a network protocol analyzer, for capturing and analyzing **HTTP (Hypertext Transfer Protocol)** traffic. Students will learn how HTTP communication occurs between a client and a web server by examining captured packets. The experiment focuses on identifying HTTP requests and responses, understanding message headers, analyzing packet details, and observing the sequence of communication during web browsing. It also provides practical exposure to packet filtering and protocol analysis, reinforcing concepts of application-layer communication.

PROCEDURE

1. Open a web browser and clear the browser cache.
2. Start Wireshark and begin packet capture.
3. Visit a website of choice.
4. Apply the display filter http to show only HTTP packets.
5. Stop the packet capture once sufficient packets are collected.
6. Analyse the first HTTP GET request and its corresponding response.

OBSERVATION METHOD**(a) Source / destination IP of the first GET message**

Select the first HTTP GET packet, expand the Internet Protocol (IP) section in the packet-details pane, and note the Source IP and Destination IP. Your own IP address can be verified in a terminal using `hostname -I`.

(b) Medium format, language, encoding and character set accepted by the client

Select the first GET packet, expand the Hypertext Transfer Protocol section, and read the Accept, Accept-Language, Accept-Encoding and Accept-Charset header fields.

(c) URL of the website and the User-Agent

Expand the Hypertext Transfer Protocol section of the first GET packet and read the Host and User-Agent header fields.

(d) Source / destination IP of the first response message

Locate the first HTTP 200 OK response packet and expand its Internet Protocol section; the source and destination addresses are reversed compared to the GET request.

(e) Status code of the first response message

Expand the Hypertext Transfer Protocol section of the response packet and read the Status Code and Reason Phrase.

(f) Last-Modified time of the HTML file

Look for the Last-Modified header field inside the response headers.

(g) Content-Length of the response

Look for the Content-Length header field inside the response headers.

(h) Time taken from GET to response

Note the Time column value for the GET packet and for the corresponding response packet, then subtract to obtain the round-trip time.

Time Taken = 4.5789 - 4.5321 = 0.0468 seconds (46.8 ms)

(i) HTTP version of the browser

Read the request line at the top of the expanded HTTP section of the GET packet - it shows the HTTP version used.

MODEL VIVA QUESTIONS

1. What is the role of the display filter http in Wireshark?
2. Differentiate between an HTTP request line and a status line.
3. What does the Content-Length header field indicate?
4. Why does the source/destination IP reverse between the request and the response?
5. What is a conditional GET, and how does Last-Modified relate to it?

RESULT

HTTP packets were successfully captured and analysed using Wireshark. The source and destination addresses, HTTP headers, response status, content information and response time were identified from the captured traffic.

EXP NO: 3**WIRESHARK-BASED ANALYSIS OF SMTP****AIM**

Compose an e-mail addressed to yourself but do not send it. Open Wireshark and start capturing, then send the e-mail using the mail client. Apply the display filter smtp, stop the capture, and save the file. Using the captured information, answer the following:

- Which of the two IP addresses seen in every SMTP packet belongs to your computer, and which to the mail server?
- Which port number belongs to the SMTP client process, and in which range does it lie?
- What is the port number of the SMTP server process?
- What do the SMTP commands / response codes observed in each packet mean?
- What is the content of the IMF (Internet Message Format) packet encapsulated inside the SMTP packet?

LEARNING OUTCOME

By the end of this experiment, learners will be able to:

1. Understand the role of SMTP in electronic mail communication.
2. Capture and filter SMTP traffic using Wireshark.
3. Identify the sequence of SMTP communication between an email client and a mail server.
4. Interpret common SMTP commands such as **HELO/EHLO, MAIL FROM, RCPT TO, DATA, and QUIT.**
5. Analyze SMTP server response codes and their significance during email transmission.
6. Examine SMTP packet headers and payloads to understand email message transfer.
7. Apply packet filtering techniques to isolate and inspect SMTP traffic.
8. Correlate captured packets with the stages of the SMTP protocol workflow.
9. Perform basic troubleshooting of email communication using packet analysis.
10. Gain practical experience in analyzing application-layer protocols and understanding the fundamentals of email transmission over computer networks.

DESCRIPTION

This experiment introduces students to the analysis of **Simple Mail Transfer Protocol (SMTP)** traffic using **Wireshark**, a network protocol analyzer. Students will capture and examine SMTP packets generated during email transmission to understand how email messages are sent from a client to a mail server. The experiment emphasizes the SMTP communication process, including connection establishment, command-response exchanges, message transfer, and session termination. Students will also learn to apply packet filters, interpret SMTP commands and response codes, and analyze email-related network traffic for troubleshooting and protocol understanding.

PROCEDURE

1. Compose an e-mail addressed to yourself.
2. Open Wireshark and start packet capturing.
3. Send the e-mail using the mail client.
4. Apply the filter smtp in Wireshark.
5. Stop the capture and save the file.
6. Analyse the captured SMTP packets.

OBSERVATION METHOD**(a) IP addresses in SMTP communication**

IP Address of Client Computer : 192.168.1.100

IP Address of SMTP Server : 74.125.24.109

The first IP address represents the local computer from which the e-mail was sent; the second represents the SMTP mail server.

(b) SMTP client port number

Client Port Number : 51432

The client port is dynamically assigned by the operating system from the ephemeral (dynamic) port range 49152-65535.

(c) SMTP server port number

SMTP Server Port Number : 25

Common SMTP ports: 25 (SMTP), 465 (SMTPS), 587 (Message Submission)

(d) SMTP commands and response codes

HELO / EHLO Identifies the client to the server

220 Service ready

MAIL FROM Specifies sender's e-mail address

250 Requested action completed successfully

RCPT TO Specifies recipient's e-mail address

DATA Begins transfer of message content

354 Start mail input; end with a single "."

QUIT Terminates the SMTP session

221 Service closing transmission channel

Sample SMTP Conversation:

220 SMTP Server Ready

EHLO client.example.com

250 OK

MAIL FROM:sender@example.com

250 OK

RCPT TO:receiver@example.com

250 OK

DATA

354 Start mail input

[Message Content].

250 Message accepted for delivery

QUIT

221 Bye

(e) IMF packet encapsulated inside the SMTP packet The IMF (Internet Message Format) packet contains the actual e-mail message: From, To, Subject, Date/Time and the Message Body.

From: student@example.com

To: student@example.com

Subject: SMTP Wireshark Experiment

This is a test e-mail sent for SMTP protocol analysis using Wireshark.

MODEL VIVA QUESTIONS

1. What is the default port used by SMTP? Name two alternative ports.
2. Explain the significance of the 354 response code.
3. What is the difference between MAIL FROM and RCPT TO?

4. What does IMF stand for, and what does it contain?
5. Is SMTP a push protocol or a pull protocol? Justify.

RESULT

SMTP communication between the client and the mail server was successfully captured and analysed using Wireshark. The IP addresses, port numbers, SMTP commands, response codes and IMF message contents were identified and studied.

EXP NO: 4**WIRESHARK-BASED ANALYSIS OF DNS****AIM**

Clear the DNS cache and the browser cache, open Wireshark and start capturing, then visit the college website. Apply the display filter dns, stop the capture, and save the file. Using the captured information, answer the following questions about the DNS query and response messages: the packet numbers, the transport protocol used, the source/destination ports and IP address, the transaction ID and its purpose, the length of the flag field, the QR bit, the bits used only in responses, and the number of question/answer/authority/additional records in the query and the response.

PROCEDURE

1. Clear the DNS cache: `sudo systemd-resolve --flush-caches` (Linux).
2. Clear the browser cache.
3. Start Wireshark packet capture.
4. Open the college website in a browser.
5. Apply the filter dns.
6. Stop the capture and save the file.
7. Analyse the DNS query and response packets.

LEARNING OUTCOME

After completing this experiment, students will be able to:

1. Understand the purpose and working of the Domain Name System (DNS).
2. Capture and analyze DNS packets using Wireshark.
3. Identify DNS query and response messages and interpret their fields.
4. Differentiate between various DNS record types such as **A, AAAA, CNAME, MX, and NS**.
5. Examine the DNS resolution process and understand how domain names are mapped to IP addresses.
6. Analyze DNS packet headers, transaction IDs, flags, and resource records.
7. Develop practical skills in network traffic monitoring and protocol analysis using Wireshark

DESCRIPTION

This experiment focuses on analyzing the **Domain Name System (DNS)** protocol using **Wireshark**, a network packet analyzer. Students capture and inspect DNS packets generated during domain name resolution. The experiment helps in understanding how DNS queries and responses are exchanged between a client and a DNS server, the structure of DNS packets, and the role of DNS in translating domain names into IP addresses.

OBSERVATION METHOD

- (a) DNS Query Packet Number : 15
- (b) Protocol Used (Query) : UDP
- (c) DNS Response Packet Number : 16 Protocol: UDP
- (d) Query - Source Port: 52543 Destination Port: 53
- (e) Response - Source Port: 53 Destination Port: 52543
- (f) DNS Server IP Address : 8.8.8.8 (Google Public DNS)
- (g) Query ID: 0x4A3B Response ID: 0x4A3B
- (h) Flag Field Length : 16 bits (2 bytes)
- (i) QR bit: 0 = Query, 1 = Response

The source port for a DNS query is an ephemeral port chosen by the client, while the destination port 53 is the standard DNS server port; the server replies from port 53 back to the client's ephemeral port. The Transaction ID uniquely identifies a query and its matching response, allowing the client to correlate them.

(j) Bits used only in the response message

AA (Authoritative Answer) - indicates whether the responding server is authoritative.

RA (Recursion Available) - indicates whether recursive queries are supported.

RCODE (Response Code) - indicates the result of the query.

0 = No Error 3 = Name Error (Domain Not Found)

(k) Records in the DNS query message

Question Records : 1 Answer Records : 0

Authority Records: 0 Additional Records: 0

(l) Records in the DNS response message

Question Records : 1 Answer Records : 1

Authority Records: 0 Additional Records: 1

The Answer Record contains the IP address corresponding to the requested domain name.

MODEL VIVA QUESTIONS

1. Why is UDP normally preferred over TCP for DNS queries?
2. What is the role of the Transaction ID field?
3. Explain the difference between a recursive and an iterative DNS query.
4. What does RCODE = 3 signify?
5. What is contained in the Additional Records section of a DNS response?

RESULT

DNS query and response packets were successfully captured and analysed using Wireshark. The DNS packet structure, transaction ID, flags, port numbers and resource records were studied and verified.

EXP NO: 5

CLIENT-SERVER COMMUNICATION USING TCP

AIM

The client inputs an integer N and creates a square matrix of order N, populated with random numbers in the range [1, 50]. It sends the matrix to the server, which identifies the matrix type (Upper Triangular, Lower Triangular or Diagonal) and returns the type, as a string, to the client for display.

LEARNING OUTCOME

Learn how to create and configure TCP sockets for client and server applications.

Implement a client-server model for reliable data transmission.

Analyze the connection establishment and termination process in TCP.

Gain hands-on experience in sending and receiving data over a network.

Develop basic debugging skills for network-based applications.

DESCRIPTION

Socket Programming in C

Socket programming is a method used in network communication that allows data to be sent and received between devices. It is a critical concept in computer networking and is widely used to develop client-server applications. the fundamentals of socket programming in C, focusing on TCP (Transmission Control Protocol) for reliable, connection-oriented communication.

Socket programming enables communication between two nodes on a network. A server listens for incoming client requests, and the client connects to the server to facilitate data exchange. Sockets provide a communication channel between two processes, either on the same machine or different machines connected via a network.

Types of Sockets

1. Stream Sockets (SOCK_STREAM):

☐ Uses TCP for communication.

- ☐ Provides reliable, connection-oriented communication.
- ☐ Data is transmitted in order and without loss.

2. Datagram Sockets (SOCK_DGRAM):

- ☐ Uses UDP (User Datagram Protocol).
- ☐ Connectionless and unreliable, but faster.
- ☐ Data may arrive out of order or get lost.

3. Raw Sockets:

- ☐ Allows direct access to lower-level protocols.

Used for packet-level manipulation.

ALGORITHM

Server

1. Start the program and create a TCP socket using `socket()`.
2. Initialize the server address structure (`AF_INET`, `INADDR_ANY`, port 8080).
3. Bind the socket to the address using `bind()`.
4. Put the socket into listening mode using `listen()`.
5. Accept a client connection using `accept()`.
6. Receive the order of the matrix `n` and the matrix itself using `read()`.
7. Traverse the matrix with nested loops, setting upper/lower/diagonal flags to 0 whenever a non-zero element is found in the corresponding region.
8. Determine the matrix type from the flags and store it as a result string.
9. Send the result string to the client using `write()` and display it.
10. Close the client and server sockets and stop.

ALGORITHM

Client

1. Start the program and create a TCP socket using `socket()`.
2. Initialize the server address structure (`AF_INET`, port 8080, IP 127.0.0.1).

3. Connect to the server using connect().
4. Read the order of the matrix n from the user.
5. Generate an n x n matrix of random numbers in [1, 50] and display it.
6. Send the matrix order and the matrix to the server using write().
7. Receive the matrix type result from the server using read() and display it.
8. Close the socket connection and stop.

PROGRAM

server

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define MAX 20
int main()
{
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    int n, matrix[MAX][MAX];
    char result[50];
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);
    bind(server_fd, (struct sockaddr *)&address, sizeof(address));
    listen(server_fd, 3);
    printf("Waiting for client connection...\n");
    new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&addrlen);
    read(new_socket, &n, sizeof(int));
    read(new_socket, matrix, sizeof(matrix));
```

```
int upper = 1, lower = 1, diagonal = 1;
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++) {
        if (i > j && matrix[i][j] != 0) upper = 0;
        if (i < j && matrix[i][j] != 0) lower = 0;
        if (i != j && matrix[i][j] != 0) diagonal = 0;
    }

if (diagonal) strcpy(result, "Diagonal Matrix");
else if (upper) strcpy(result, "Upper Triangular Matrix");
else if (lower) strcpy(result, "Lower Triangular Matrix");
else strcpy(result, "Neither Upper nor Lower nor Diagonal");
write(new_socket, result, sizeof(result));
printf("Matrix Type Sent to Client: %s\n", result);
close(new_socket);
close(server_fd);
return 0;
}
```

client

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define MAX 20

int main()
{
    int sock = 0;
    struct sockaddr_in serv_addr;
    int n, matrix[MAX][MAX];
    char result[50];
    sock = socket(AF_INET, SOCK_STREAM, 0);
    serv_addr.sin_family = AF_INET;
```

```
serv_addr.sin_port = htons(PORT);
inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);
connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
printf("Enter order of matrix: ");
scanf("%d", &n);
srand(time(0));
printf("\nGenerated Matrix:\n");
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        matrix[i][j] = rand() % 50 + 1;
        printf("%4d", matrix[i][j]);
    }
    printf("\n");
}
write(sock, &n, sizeof(int));
write(sock, matrix, sizeof(matrix));
read(sock, result, sizeof(result));
printf("\nMatrix Type Received from Server: %s\n", result);
close(sock);
return 0;
}
```

EXPECTED OUTPUT

student@cse-hwl-030:~/surya/socket pgm/c\$ cc server.c

student@cse-hwl-030:~/surya/socket pgm/c\$./a.out

Waiting for client connection...

Matrix Type Sent to Client: Neither Upper nor Lower nor Diagonal

student@cse-hwl-030:~/surya/socket pgm/c\$ cc client.c

student@cse-hwl-030:~/surya/socket pgm/c\$./a.out

Enter order of matrix: 3

Generated Matrix:

8 16 2

21 33 14

9 48 32

Matrix Type Received from Server: Neither Upper nor Lower nor Diagonal

MODEL VIVA QUESTIONS

1. What is the difference between connection-oriented and connectionless sockets?
2. Explain the purpose of bind(), listen() and accept() on the server side.
3. Why must the client call connect() before exchanging data?
4. How is a diagonal matrix distinguished from an upper/lower triangular matrix in this program?
5. What happens if the server calls accept() before the client calls connect()?

RESULT

A TCP client-server application was implemented and executed successfully. The server correctly identified the type of the matrix sent by the client and returned the result, which was displayed at the client.

EXP NO: 6

CLIENT–SERVER COMMUNICATION USING UDP – SLANG TO FORMAL ENGLISH TRANSLATOR

AIM:

To design a client–server application using UDP sockets in which the client accepts a sentence written in informal, abbreviation-heavy English (e.g., tbh, idc, ig, tbf, atm, irl, lol, asap, omg, ttyl, idk, nvm) and sends it to the server. The server translates the sentence into formal English and returns the translated sentence to the client, which then displays it.

LEARNING OUTCOMES:

Upon completion of this lab, the student will be able to:

- Create and use UDP sockets for connectionless communication.
- Use `sendto()` and `recvfrom()` for datagram-based data exchange.
- Parse and tokenize a string using `strtok()` in C.
- Implement simple text-processing logic on the server side.

DESCRIPTION

This experiment demonstrates **client–server communication using the User Datagram Protocol (UDP)** by implementing a Slang to Formal English Translator. The client sends a slang word or phrase to the server, and the server searches its predefined dictionary to find the corresponding formal English equivalent. The translated response is then sent back to the client using UDP datagrams. This experiment helps students understand the connectionless communication model of UDP, message transmission, and basic client–server application development in computer networks. It also provides practical experience in socket programming, data exchange, and handling user requests over a network without establishing a dedicated connection.

ALGORITHM:

Server

1. Start the program.
2. Create a UDP socket using `socket(AF_INET, SOCK_DGRAM, 0)`.
3. Initialize the server address structure: set address family to `AF_INET`, IP address to `INADDR_ANY`, and port number to 8080.
4. Bind the socket to the server address using `bind()`.
5. Display "UDP Server Running...".
6. Wait for a message from the client using `recvfrom()`.
7. Receive the sentence and store it in a buffer; display the received sentence.
8. Call the `translate()` function: split the sentence into words using `strtok()`, convert each word to lowercase, check whether the word is a known abbreviation, and replace abbreviations with their full forms (tbh->to be honest, idc->I don't care, ig->I guess, tbf->to be fair, atm->at the moment, irl->in real life, lol->laughing out loud, asap->as soon as possible, omg->oh my God, ttyl->talk to you later, idk->I don't know, nvm->never mind); leave other words unchanged and form the translated sentence.
9. Send the translated sentence back to the client using `sendto()`.
10. Display "Translated message sent."
11. Close the socket.
12. Stop the program.

Client

1. Start the program.
2. Create a UDP socket using `socket(AF_INET, SOCK_DGRAM, 0)`.
3. Initialize the server address structure with address family `AF_INET`, port number 8080, and the server IP address.
4. Prompt the user to enter a sentence and read it using `fgets()`.
5. Send the sentence to the server using `sendto()`.
6. Wait for the translated response from the server using `recvfrom()`.
7. Receive and display the translated sentence.
8. Close the socket.
9. Stop the program.

PROCEDURE:

1. Open a Linux terminal.

2. Type in the server program in a file named `udpserver.c` and the client program in a file named `udpclient.c`, as given below.

3. Compile the server program: `gcc udpserver.c -o udpserver`

4. Compile the client program: `gcc udpclient.c -o udpclient`

5. Run the server in one terminal: `./udpserver`

6. Run the client in a second terminal: `./udpclient`

7. At the client prompt, type a sentence containing informal abbreviations and press Enter.

8. Observe the translated sentence printed by the client, and the corresponding log messages printed by the server.

9. Repeat with different sentences and abbreviation combinations to verify correctness.

PROGRAM

Server

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define MAX 1024
void translate(char *str)
{
    char temp[MAX];
    char result[MAX] = "";
    char *token;
    strcpy(temp, str);
    token = strtok(temp, " ,!?\n");
```

```
while (token != NULL) {  
    char word[50];  
    strcpy(word, token);  
    for (int i = 0; word[i]; i++) word[i] = tolower(word[i]);  
    if (strcmp(word, "tbh") == 0) strcat(result, "to be honest");  
    else if (strcmp(word, "idc") == 0) strcat(result, "I don't care");  
    else if (strcmp(word, "ig") == 0) strcat(result, "I guess");  
    else if (strcmp(word, "tbf") == 0) strcat(result, "to be fair");  
    else if (strcmp(word, "atm") == 0) strcat(result, "at the moment");  
    else if (strcmp(word, "irl") == 0) strcat(result, "in real life");  
    else if (strcmp(word, "lol") == 0) strcat(result, "laughing out loud");  
    else if (strcmp(word, "asap") == 0) strcat(result, "as soon as possible");  
    else if (strcmp(word, "omg") == 0) strcat(result, "oh my God");  
    else if (strcmp(word, "ttyl") == 0) strcat(result, "talk to you later");  
    else if (strcmp(word, "idk") == 0) strcat(result, "I don't know");  
    else if (strcmp(word, "nvm") == 0) strcat(result, "never mind");  
    else strcat(result, token);  
    strcat(result, " ");  
    token = strtok(NULL, " ,!?\n");  
}  
strcpy(str, result);  
}  
  
int main()  
{  
    int sockfd;  
    char buffer[MAX];  
    struct sockaddr_in server_addr, client_addr;  
    socklen_t len = sizeof(client_addr);
```

```
sockfd = socket(AF_INET, SOCK_DGRAM, 0);

server_addr.sin_family = AF_INET;

server_addr.sin_addr.s_addr = INADDR_ANY;

server_addr.sin_port = htons(PORT);

bind(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr));

printf("UDP Server Running...\n");

recvfrom(sockfd, buffer, MAX, 0, (struct sockaddr *)&client_addr, &len);

printf("Received: %s\n", buffer);

translate(buffer);

sendto(sockfd, buffer, strlen(buffer) + 1, 0, (struct sockaddr *)&client_addr, len);

printf("Translated message sent.\n");

close(sockfd);

return 0;

}
```

client

```
#include <stdio.h>

#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>

#define PORT 8080

#define MAX 1024

int main()

{

int sockfd;

char message[MAX];

char buffer[MAX];

struct sockaddr_in server_addr;

socklen_t len = sizeof(server_addr);
```

```
sockfd = socket(AF_INET, SOCK_DGRAM, 0);

server_addr.sin_family = AF_INET;

server_addr.sin_port = htons(PORT);

server_addr.sin_addr.s_addr = INADDR_ANY;

printf("Enter sentence:\n");

fgets(message, MAX, stdin);

sendto(sockfd, message, strlen(message) + 1, 0, (struct sockaddr *)&server_addr, len);

recvfrom(sockfd, buffer, MAX, 0, (struct sockaddr *)&server_addr, &len);

printf("\nTranslated Sentence:\n%s\n", buffer);

close(sockfd);

return 0;

}
```

OUTPUT

```
$ cc udpserver.c
```

```
$ ./a.out
```

UDP Server Running...

Received: Really idc about this stupid server as it is of no use irl but atm I will design one tbf to the professor

Translated message sent.

```
$ cc udpclient.c
```

```
$ ./a.out
```

Enter sentence:

Really idc about this stupid server as it is of no use irl but atm I will design one tbf to the professor

Translated Sentence:

Really I don't care about this stupid server as it is of no use in real life but at the moment I will design one to be fair to the professor

MODEL VIVA QUESTIONS:

1. What is the difference between UDP and TCP sockets?
2. Why is UDP called a connectionless protocol?
3. What is the purpose of the `sendto()` and `recvfrom()` system calls?
4. Why does a UDP server not call `listen()` or `accept()`, unlike a TCP server?
5. What is the role of the `strtok()` function in the `translate()` logic?
6. What happens if the client sends a sentence larger than the buffer size?
7. Is UDP reliable? What could go wrong if a packet is lost in this application?

RESULT

A UDP client-server application was implemented and executed successfully. The server correctly translates the sentence into formal English and returns the translated sentence to the client, which then displays it.

EXP NO:7

MULTI-USER CHAT SERVER USING TCP

AIM:

To implement a multi-user chat server using TCP as the transport layer protocol, so that multiple clients can connect to a common server and exchange text messages, with every message broadcast to all other connected clients.

LEARNING OUTCOMES:

Upon completion of this lab, the student will be able to:

- Create a concurrent, multi-client TCP server using threads.
- Use `pthread_create()` to service each client connection independently.
- Use a mutex (`pthread_mutex_t`) to protect a shared client list during broadcast.
- Implement simultaneous, bidirectional communication using a dedicated receiver thread on the client side.

DESCRIPTION

This experiment demonstrates the implementation of a **multi-user chat application using the Transmission Control Protocol (TCP)**. A TCP server is designed to accept connections from multiple clients simultaneously, enabling them to exchange messages in real time. The server manages client connections, receives messages from connected clients, and broadcasts them to all other participants in the chat session. Through this experiment, students gain practical experience in TCP socket programming, concurrent client handling, reliable data transmission, and the design of network-based communication applications. It also highlights the advantages of TCP, such as connection-oriented communication, ordered data delivery, and error-free message transmission, making it suitable for real-time collaborative applications.

ALGORITHM:**Server**

1. Start the program and initialize a mutex lock using `pthread_mutex_init()`.
2. Create a TCP socket using `socket()`.
3. Configure the server address: address family `AF_INET`, IP address `INADDR_ANY`, port number 8080.
4. Bind the socket to the server address using `bind()` and put it into listening mode using `listen()`.
5. Display "Server running on port 8080".
6. Enter an infinite loop: wait for a client connection using `accept()`, store the client socket in the client list, increment the client count, and create a new thread using `pthread_create()` to run `handle_client()` for that client.
7. In `handle_client()`: continuously receive messages from the client using `recv()`; on receiving a message, null-terminate it and call `broadcast()` to send it to every other connected client; on disconnection, close the client socket and terminate the thread.
8. In `broadcast()`: lock the mutex, traverse the client list, send the message to every client except the sender, and unlock the mutex.
9. Continue serving multiple clients until the server is terminated, then close the server socket.

Client

1. Start the program and create a TCP socket using `socket()`.
2. Configure the server address: address family `AF_INET`, port number 8080, server IP 127.0.0.1.
3. Connect to the server using `connect()`.
4. Create a separate thread using `pthread_create()` to run `receive_msg()`, which continuously receives messages from the server using `recv()` and displays them.
5. In the main thread, continuously read user input using `fgets()` and send each message to the server using `send()`.
6. Repeat until the client exits, then close the socket.
8. Observe that the message appears on every other connected client's terminal, but not on the sender's own terminal.
9. Repeat from multiple clients to confirm that the server broadcasts correctly to all participants.

PROGRAM

server

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>

#include <pthread.h>

#define PORT 8080

#define MAX_CLIENTS 10

#define BUFFER_SIZE 1024

int clients[MAX_CLIENTS];

int client_count = 0;

pthread_mutex_t lock;

void broadcast(char *message, int sender)

{

pthread_mutex_lock(&lock);

for (int i = 0; i < client_count; i++) {

if (clients[i] != sender)

send(clients[i], message, strlen(message), 0);

}

pthread_mutex_unlock(&lock);

}

void *handle_client(void *arg)

{

int client_socket = *((int *)arg);

char buffer[BUFFER_SIZE];

int len;

while ((len = recv(client_socket, buffer, BUFFER_SIZE, 0)) > 0) {
```

```
buffer[len] = '\0';

broadcast(buffer, client_socket);

}

close(client_socket);

pthread_exit(NULL);

}

int main()
{
    int server_fd, client_socket;

    struct sockaddr_in server_addr, client_addr;

    socklen_t addr_size;

    pthread_mutex_init(&lock, NULL);

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    server_addr.sin_family = AF_INET;

    server_addr.sin_port = htons(PORT);

    server_addr.sin_addr.s_addr = INADDR_ANY;

    bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr));

    listen(server_fd, 5);

    printf("Server running on port %d\n", PORT);

    while (1) {
        addr_size = sizeof(client_addr);

        client_socket = accept(server_fd, (struct sockaddr*)&client_addr, &addr_size);

        clients[client_count++] = client_socket;

        pthread_t tid;

        pthread_create(&tid, NULL, handle_client, &clients[client_count - 1]);

    }

    close(server_fd);
```

```
return 0;

}

client

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>

#include <pthread.h>

#define PORT 8080

#define BUFFER_SIZE 1024

int sockfd;

void *receive_msg(void *arg)

{

    char message[BUFFER_SIZE];

    while (recv(sockfd, message, BUFFER_SIZE, 0) > 0) {

        printf("%s", message);

    }

    return NULL;

}

int main(){

    struct sockaddr_in server_addr;

    char message[BUFFER_SIZE];

    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    server_addr.sin_family = AF_INET;

    server_addr.sin_port = htons(PORT);

    inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);

    connect(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr));
```

```
pthread_t tid;

pthread_create(&tid, NULL, receive_msg, NULL);

while (fgets(message, BUFFER_SIZE, stdin)) {
    send(sockfd, message, strlen(message), 0);
}close(sockfd);

return 0;}
```

EXPECTED OUTPUT

```
$ cc chatserver.c
```

```
$ ./a.out
```

Server running on port 8080

```
$ cc chatclient.c (Terminal 1)
```

```
$ ./a.out
```

hello

hi

how r u

```
$ cc chatclient.c (Terminal 2)
```

```
$ ./a.out
```

hello

hi

how r u

MODEL VIVA QUESTIONS:

1. Why is TCP preferred over UDP for a chat application?
2. What is the purpose of creating a new thread for each client using `pthread_create()`?
3. Why is a mutex (`pthread_mutex_t`) required while broadcasting messages to clients?
4. What would happen if the mutex lock/unlock around `broadcast()` were removed?
5. Why does the client need a separate thread to receive messages while the main thread reads user input?

RESULT:

A multi chat server application was implemented and executed successfully.

EXP NO: 8**IMPLEMENTATION OF A CONCURRENT TIME SERVER USING UDP****AIM:**

To implement a concurrent Time Server application using UDP, where the client sends a time request to a remote server, the server replies with its current system time, and the client displays the received time value.

LEARNING OUTCOME

- Understand the working principles of the **User Datagram Protocol (UDP)** and its connectionless communication model.
- Implement a UDP-based concurrent server capable of serving multiple client requests.
- Develop client and server applications using UDP socket programming.
- Understand the process of sending and receiving datagrams between network hosts.
- Demonstrate how a server responds to multiple client requests with the current system date and time.
- Analyze the advantages and limitations of UDP for lightweight, real-time network applications.
- Gain practical experience in designing and testing network-based client–server applications using UDP.

DESCRIPTION

This experiment demonstrates the implementation of a **concurrent time server using the User Datagram Protocol (UDP)**. The server listens for time requests from multiple clients and responds with the current system date and time without establishing a dedicated connection. Since UDP is a connectionless protocol, the server can efficiently handle requests from multiple clients independently by processing each incoming datagram and sending the corresponding response. This experiment provides practical experience in UDP socket programming, concurrent request handling,

and the development of lightweight network services for time synchronization and information retrieval.

ALGORITHM:

Server

1. Start the program.
2. Create a UDP socket using `socket(AF_INET, SOCK_DGRAM, 0)`.
3. Initialize the server address structure by setting the address family to `AF_INET`, the IP address to `INADDR_ANY`, and the port number to 8080.
4. Bind the socket to the server address using `bind()`.
5. Display the message "UDP Time Server Running...".
6. Enter an infinite loop.
7. Wait for a request from a client using `recvfrom()`.
8. On receiving a request, obtain the current system time using `time()`, convert it to a readable string using `ctime()`, and copy the formatted string into the buffer.
9. Send the current date and time back to the client using `sendto()`.
10. Repeat steps 7-9 for every client request.
11. Close the socket when the server terminates.
12. Stop.

Client

1. Start the program.
2. Create a UDP socket using `socket(AF_INET, SOCK_DGRAM, 0)`.
3. Initialize the server address structure with address family `AF_INET`, port number 8080, and server IP address 127.0.0.1.
4. Store the request message "TIME" in a buffer.
5. Send the request message to the server using `sendto()`.
6. Wait for the server's response using `recvfrom()`.

7. Receive the current date and time from the server.

8. Display the received time on the screen.

9. Close the socket.

10. Stop.

PROGRAM:

Server

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>

#include <time.h>

#define PORT 8080

#define BUFFER_SIZE 1024

int main()

{

int sockfd;

char buffer[BUFFER_SIZE];

struct sockaddr_in server_addr, client_addr;

socklen_t len;

sockfd = socket(AF_INET, SOCK_DGRAM, 0);

server_addr.sin_family = AF_INET;

server_addr.sin_addr.s_addr = INADDR_ANY;

server_addr.sin_port = htons(PORT);

bind(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr));

printf("UDP Time Server Running...\n");

while(1)
```

```
{  
len = sizeof(client_addr);  
recvfrom(sockfd, buffer, BUFFER_SIZE, 0, (struct sockaddr *)&client_addr, &len);  
time_t current_time;  
time(&current_time);  
strcpy(buffer, ctime(&current_time));  
sendto(sockfd, buffer, strlen(buffer), 0, (struct sockaddr *)&client_addr, len);  
}  
close(sockfd);  
return 0;  
}
```

Client

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <unistd.h>  
#include <arpa/inet.h>  
#define PORT 8080  
#define BUFFER_SIZE 1024  
int main()  
{  
int sockfd;  
char buffer[BUFFER_SIZE] = "TIME";  
struct sockaddr_in server_addr;  
socklen_t len;  
sockfd = socket(AF_INET, SOCK_DGRAM, 0);  
server_addr.sin_family = AF_INET;  
server_addr.sin_port = htons(PORT);
```

```
server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");  
sendto(sockfd, buffer, strlen(buffer), 0, (struct sockaddr *)&server_addr, sizeof(server_addr));  
len = sizeof(server_addr);  
recvfrom(sockfd, buffer, BUFFER_SIZE, 0, (struct sockaddr *)&server_addr, &len);  
printf("Server Time: %s\n", buffer);  
close(sockfd);  
return 0;  
}
```

EXPECTED OUTPUT

student@cse-hwl-030:~/surya/socket pgm/c\$ cc timeserver.c

student@cse-hwl-030:~/surya/socket pgm/c\$./a.out

UDP Time Server Running...

student@cse-hwl-030:~/surya/socket pgm/c\$ cc timeclient.c

student@cse-hwl-030:~/surya/socket pgm/c\$./a.out

Server Time: Thu Jun 18 11:18:36 2026

MODEL VIVA QUESTIONS

1. What is the difference between TCP and UDP, and why is UDP used in this experiment?
2. What is the purpose of the `recvfrom()` and `sendto()` system calls?
3. Why does a UDP server not need to call `listen()` or `accept()`, unlike a TCP server?
4. What do the `time()` and `ctime()` library functions do?
5. What is the role of the `sockaddr_in` structure in socket programming?
6. What does the term "concurrent" mean in the context of this Time Server, and how could true concurrency be added if multiple clients requested time simultaneously?

RESULT:

A concurrent Time Server application using UDP was implemented and executed successfully. The client's request for the current time was received by the server, which replied with its system time, and the client displayed the received time correctly.

EXP NO: 9**DEVELOPMENT OF A CONCURRENT FILE SERVER****AIM:**

To develop a concurrent file server that provides the file requested by the client if it exists, and sends an appropriate error message if the file does not exist. The server also sends its process ID (PID) to the client, to be displayed along with the file contents or the error message.

LEARNING OUTCOME

- Understand the architecture and working of a concurrent file server.
- Implement client–server communication using socket programming.
- Develop a server capable of handling multiple client requests concurrently.
- Apply file handling techniques to manage file access, transfer, and retrieval.
- Analyze the role of concurrency in improving server performance and resource utilization.
- Understand synchronization and efficient management of multiple client connections.
- Gain practical experience in designing, implementing, testing, and debugging concurrent network applications.

DESCRIPTION

This experiment focuses on the development of a **concurrent file server** that can handle multiple client requests simultaneously over a network. The server accepts connections from multiple clients, processes their file-related requests such as retrieving or downloading files, and responds concurrently using separate processes or threads. This experiment provides hands-on experience in implementing client–server communication, concurrent programming, file handling, and socket programming. It also demonstrates the importance of concurrency in improving the performance, scalability, and responsiveness of network-based file services.

ALGORITHM**Server**

1. Start.
2. Create a TCP socket using `socket()`.
3. Initialize the server address structure by setting the address family to `AF_INET`, the IP address to `INADDR_ANY`, and the port number to 8080.
4. Bind the socket to the server address using `bind()`.
5. Put the socket into listening mode using `listen()`.
6. Display the message "File Server Running...".
7. Enter an infinite loop.
8. Accept a client connection using `accept()`.
9. Create a child process using `fork()` to handle the client concurrently.
10. In the child process, close the listening socket, receive the requested filename from the client using `recv()`, and attempt to open the file using `fopen()`.
11. Send the child process's PID to the client.
12. If the file does not exist, send the message "Requested file does not exist." to the client. Otherwise, read the file line by line using `fgets()` and send each line to the client using `send()`, then close the file.
13. Close the client socket and terminate the child process.
14. In the parent process, close the connected client socket and continue waiting for new clients.
15. Repeat steps 8-14 for every client request.
16. Stop when the server is terminated.

Client

1. Start.
2. Create a TCP socket using `socket()`.
3. Initialize the server address structure with address family `AF_INET`, port number 8080, and server IP address 127.0.0.1.

4. Connect to the server using connect().
5. Prompt the user to enter a filename and read it from the keyboard.
6. Send the filename to the server using send().
7. Display the message "Response from Server:".
8. Continuously receive data from the server using recv() and print it on the screen, until the server closes the connection.
9. Close the socket.
10. Stop.

PROGRAM

Server

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>

#define PORT 8080

#define BUFFER_SIZE 1024

int main()
{
    int server_fd, client_socket;

    struct sockaddr_in server_addr, client_addr;

    socklen_t addr_len;

    char filename[100];

    char buffer[BUFFER_SIZE];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    server_addr.sin_family = AF_INET;

    server_addr.sin_addr.s_addr = INADDR_ANY;

    server_addr.sin_port = htons(PORT);
```

```
bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));

listen(server_fd, 5);

printf("File Server Running...\n");

while(1)
{
    addr_len = sizeof(client_addr);
    client_socket = accept(server_fd, (struct sockaddr *)&client_addr, &addr_len);
    if(fork() == 0)
    {
        close(server_fd);
        recv(client_socket, filename, sizeof(filename), 0);
        FILE *fp = fopen(filename, "r");
        sprintf(buffer, "Server PID: %d\n\n", getpid());
        send(client_socket, buffer, strlen(buffer), 0);
        if(fp == NULL)
        {
            strcpy(buffer, "Requested file does not exist.\n");
            send(client_socket, buffer, strlen(buffer), 0);
        }else{
            while(fgets(buffer, BUFFER_SIZE, fp) != NULL)
            {
                send(client_socket, buffer, strlen(buffer), 0);
            }
            fclose(fp);
        }
        close(client_socket);
        exit(0);
    }
    close(client_socket);
}
```

```
}  
  
return 0;  
  
}
```

Client Side

```
#include <stdio.h>  
  
#include <stdlib.h>  
  
#include <string.h>  
  
#include <unistd.h>  
  
#include <arpa/inet.h>  
  
#define PORT 8080  
  
#define BUFFER_SIZE 1024  
  
int main()  
{  
  
int sockfd;  
  
struct sockaddr_in server_addr;  
  
char filename[100];  
  
char buffer[BUFFER_SIZE];  
  
sockfd = socket(AF_INET, SOCK_STREAM, 0);  
  
server_addr.sin_family = AF_INET;  
  
server_addr.sin_port = htons(PORT);  
  
server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");  
  
connect(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr));  
  
printf("Enter filename: ");  
  
scanf("%s", filename);  
  
send(sockfd, filename, strlen(filename) + 1, 0);  
  
printf("\nResponse from Server:\n\n");  
  
while(recv(sockfd, buffer, BUFFER_SIZE - 1, 0) > 0)
```

```
{  
buffer[BUFFER_SIZE - 1] = '\0';  
printf("%s", buffer);  
}  
close(sockfd);  
return 0;  
}
```

sample.txt

welcome to networks lab

EXPECTED OUTPUT

student@cse-hwl-030:~/surya/socket pgm/c\$ cc fileserver.c

student@cse-hwl-030:~/surya/socket pgm/c\$./a.out

File Server Running...

student@cse-hwl-030:~/surya/socket pgm/c\$ cc fileclient.c

student@cse-hwl-030:~/surya/socket pgm/c\$./a.out

Enter filename: test.txt

Response from Server:

Server PID: 7779

Requested file does not exist.

student@cse-hwl-030:~/surya/socket pgm/c\$ cc fileclient.c

student@cse-hwl-030:~/surya/socket pgm/c\$./a.out

Enter filename: sample.txt

Response from Server:

Server PID: 7792

welcome to networks lab

MODEL VIVA QUESTIONS

1. Why is fork() used in this server, and what problem would arise if it were removed?

2. What is the difference between the parent process and the child process after a `fork()` call?
3. Why must the server close the listening socket (`server_fd`) inside the child process?
4. How does the client know when the server has finished sending the file contents?
5. What is the purpose of sending the server's PID to the client?
6. What would happen if two clients requested the same file at the same time? Explain how this server's design supports that scenario.

RESULT

A concurrent file server was successfully implemented and tested. The server correctly served requested files that existed, along with its process ID, and returned an appropriate error message for files that did not exist, thereby demonstrating concurrent client handling using `fork()`.

EXP NO: 10**PACKET-CAPTURING APPLICATION USING RAW SOCKETS****AIM:**

To develop a packet-capturing application using raw sockets that captures Ethernet packets arriving at a network interface in real time and displays the packet number, packet size, source IP address, and destination IP address.

LEARNING OUTCOMES

- Understand the concept and purpose of raw sockets in Linux network programming.
- Learn to capture raw Ethernet frames using the AF_PACKET address family.
- Understand the structure of an Ethernet frame and an IP header.
- Learn to extract source and destination IP addresses from a captured packet using the iphdr structure.
- Understand why raw socket programs require superuser (root) privileges.
- Appreciate how tools such as tcpdump and Wireshark make use of raw sockets internally.

DESCRIPTION

A raw socket is a special type of socket that allows a program to bypass the usual transport-layer (TCP/UDP) processing of the operating system and directly send or receive packets at a lower level, including the underlying Ethernet frame. In Linux, a raw socket capable of capturing every packet arriving at an interface is created using `socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL))`. The AF_PACKET family operates at the device driver (data link) layer, and ETH_P_ALL indicates that packets of every protocol type should be captured, not just IP.

Each captured packet begins with a 14-byte Ethernet header, containing the destination MAC address, source MAC address, and the EtherType field. Once this header is skipped, the remaining bytes can be interpreted as an IP header by casting them to a struct iphdr, which is defined in `<netinet/ip.h>`.

This structure exposes fields such as `saddr` (source address) and `daddr` (destination address) in binary network-byte-order form, which are converted to a human-readable dotted-decimal string using the library function `inet_ntoa()`. Because raw sockets provide unrestricted access to network traffic, creating one requires the program to be run with root privileges; this same mechanism underlies packet-sniffing and network-monitoring tools such as `tcpdump` and `Wireshark`.

ALGORITHM

1. Start.
2. Create a raw socket using `socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL))`; this allows capturing all Ethernet packets.
3. Check whether the socket was created successfully; if socket creation fails, display an error message and terminate the program.
4. Allocate memory for a packet buffer of size 65536 bytes.
5. Display the message "Packet Capture Started...".
6. Initialize the packet counter to 0.
7. Enter an infinite loop.
8. Receive a packet from the network using `recvfrom()`.
9. Check whether packet reception was successful; if an error occurs, display an error message and exit the loop.
10. Extract the IP header from the received packet by skipping the 14-byte Ethernet header and casting the remaining data to an IP header structure (`iphdr`).
11. Obtain the source IP address (`saddr`) and destination IP address (`daddr`) from the IP header.
12. Convert the IP addresses into readable dotted-decimal format using `inet_ntoa()`.
13. Increment the packet counter.
14. Display the packet number, packet size, source IP address, and destination IP address.
15. Display a separator line.
16. Repeat steps 8-15 for every incoming packet.
17. When the program terminates, close the raw socket and free the allocated memory.
18. Stop.

PROGRAM:

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <arpa/inet.h>

#include <sys/socket.h>

#include <linux/if_packet.h>

#include <linux/if_ether.h>

#include <netinet/ip.h>

#define BUFFER_SIZE 65536

int main() {

int sock_raw;

unsigned char *buffer;

int packet_count = 0;

sock_raw = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));

if(sock_raw < 0) {

perror("Socket Error"); return 1; }

buffer = (unsigned char *)malloc(BUFFER_SIZE);

printf("Packet Capture Started...\n");

while(1) {

int data_size;

data_size = recvfrom(sock_raw, buffer, BUFFER_SIZE, 0, NULL, NULL);

if(data_size < 0) {

perror("Receive Error"); break; }

struct iphdr *ip_header = (struct iphdr *)(buffer + 14);

struct sockaddr_in src, dst; src.sin_addr.s_addr = ip_header->saddr;

dst.sin_addr.s_addr = ip_header->daddr;

printf("\nPacket No: %d\n", ++packet_count);
```

```
printf("Packet Size : %d bytes\n", data_size);  
printf("Source IP : %s\n", inet_ntoa(src.sin_addr));  
printf("Destination IP : %s\n", inet_ntoa(dst.sin_addr));  
printf("-----\n"); }  
close(sock_raw);  
free(buffer);  
return 0; }
```

EXPECTED OUTPUT

cse@cse-hwl-030:~\$ cc packet.c

cse@cse-hwl-030:~\$ sudo ./a.out

[sudo] password for cse:

Packet Capture Started...

Packet No: 1

Packet Size : 87 bytes

Source IP : 10.100.74.66

Destination IP : 224.0.0.251

Packet No: 2

Packet Size : 87 bytes

Source IP : 10.100.74.66

Destination IP : 224.0.0.251

Packet No: 3

Packet Size : 60 bytes

Source IP : 88.60.192.168

Destination IP : 0.239.0.0

MODEL VIVA QUESTIONS:

1. What is a raw socket, and how does it differ from a normal stream or datagram socket?
2. Why does this program need to be run with sudo / root privileges?
3. What does the AF_PACKET address family represent, and where does it operate in the network stack?
4. What is the purpose of specifying htons(ETH_P_ALL) in the socket() call?
5. Why is the first 14 bytes of the received buffer skipped before reading the IP header?
6. What do the saddr and daddr fields of the struct iphdr represent?
7. What does the function inet_ntoa() do, and why is it needed here?
8. Name two real-world tools that rely on the same raw-socket packet-capturing mechanism.

RESULT

A packet-capturing application using raw sockets was successfully developed and executed. All Ethernet packets arriving at the network interface were captured in real time, and for each captured packet the packet number, packet size, source IP address, and destination IP address were correctly displayed.

EXP NO: 11**FAMILIARIZING ROUTER COMMANDS****AIM**

To familiarize with basic Cisco router commands, router modes, interface configuration, routing information, and configuration management.

LEARNING OUTCOMES

- Understand the different operating modes of a Cisco router: User EXEC mode, Privileged EXEC mode, and Global Configuration mode.
- Learn to use show commands to view router hardware and software information, routing protocols, and the routing table.
- Learn to save the router's running configuration and view the command history.
- Understand how to view the router clock and the list of known hosts.
- Learn to display interface statistics in both detailed and brief formats.
- Understand the difference between a DTE (Data Terminal Equipment) device and a DCE (Data Communication Equipment) device on a serial link.
- Learn to configure Ethernet and serial interfaces with an IP address, subnet mask, and clock rate.

DESCRIPTION:

A Cisco router's command-line interface (CLI) is organized into a hierarchy of modes, each offering a different level of access. User EXEC mode (Router>) is the default mode on login and allows only basic, non-destructive commands such as viewing limited status information. Privileged EXEC mode (Router#), entered using the enable command, allows the use of powerful show and debugging commands as well as access to configuration modes. Global Configuration mode (Router(config)#), entered using configure terminal, is used to change device-wide settings, while Interface

Configuration mode (Router(config-if)#) is used to configure a specific interface, such as assigning an IP address.

The router maintains a large amount of operational information that can be viewed using show commands: show version displays hardware and IOS details, show ip route displays the routing table, show ip protocols displays configured routing protocols, show interfaces and show ip interface brief display interface statistics, and show controllers reveals whether a serial interface is acting as DTE or DCE. On a serial link between two routers, the DCE end supplies the clock signal (configured using the clock rate command) while the DTE end derives its timing from the DCE. Configuration changes made in RAM (the running-config) are temporary until saved to NVRAM (the startup-config) using the copy running-config startup-config command, ensuring the configuration survives a reboot.

ALGORITHM

1. Access the router through the console.
2. Identify the current router mode (User EXEC or Privileged EXEC).
3. Enter Privileged EXEC mode using the enable command.
4. Enter Global Configuration mode using the configure terminal.
5. Display router hardware and software information.
6. Check the status of configured routing protocols.
7. View the routing table.
8. Save the current running configuration.
9. Display previously executed commands.
10. View the router clock and host table.
11. Display detailed and brief interface statistics.
12. Check whether the serial interface is DTE or DCE.
13. Configure Ethernet and serial interfaces with IP addresses and clock rate.
14. Verify the configuration using appropriate show commands.

PROGRAM

As this experiment involves configuring a Cisco router rather than writing a host program, the "program" consists of the sequence of IOS commands entered at the router CLI, summarized below.

```
Router>enable
Router#configure terminal
Router(config)# exit
Router# disable
Router# show version
Router# show interfaces
Router# show ip protocols
Router# show ip route
Router# copy running-config startup-config
Router# show history
Router# show clock
Router# show hosts
Router# show controllers serial 0/0/0
Router# configure terminal
Router(config)# interface fastethernet0/0
Router(config-if)# ip address 192.168.1.1 255.255.255.0
Router(config-if)# no shutdown
Router(config-if)# exit
Router(config)# interface serial0/0/0
Router(config-if)# ip address 10.0.0.1 255.0.0.0
Router(config-if)# clock rate 64000
Router(config-if)# no shutdown
Router(config-if)# end
Router# show ip interface brief
```

PROCEDURE:

(a) Knowing the current mode and switching to Privileged mode

```
Router>enable
```

```
Router#
```

To return to User mode: Router# disable

(b) Switching to configuration mode

```
Router# configure terminal
```

```
Router(config)#
```

(c) Obtaining router information (type, OS, memory, interfaces)

```
Router# show version
```

Cisco IOS Software, Version 15.1

Router Model: 2811

Processor Board ID FTX123456

256 MB RAM

System image file: flash:c2800nm.bin

The command show interfaces displays interface names, MAC addresses, MTU size, bandwidth, and packet statistics for every interface.

(d) Viewing the status of configured routing protocols

```
Router# show ip protocols
```

Routing Protocol is "rip"

Sending updates every 30 seconds

Routing for Networks:

192.168.1.0

10.0.0.0

(e) Showing the routing table

```
Router# show ip route
```

C 192.168.1.0/24 is directly connected, FastEthernet0/0

C 10.0.0.0/8 is directly connected, Serial0/0/0

(f) Saving the running configuration

```
Router# copy running-config startup-config
```

```
Destination filename [startup-config]?
```

```
Building configuration...
```

```
[OK]
```

(g) Viewing the command history

```
Router# show history
```

```
enable
```

```
configure terminal
```

```
show version
```

```
show ip route
```

```
show history
```

(h) Viewing the router clock

```
Router# show clock
```

```
14:25:30.123 UTC Mon Mar 10 2025
```

(i) Viewing the list of hosts

```
Router# show hosts
```

```
Default domain is not set
```

```
Name Servers are 8.8.8.8
```

```
Host      Address
```

```
Server1   192.168.1.100
```

```
PC1       192.168.1.101
```

(j) Displaying statistics for all interfaces (detailed and brief)

```
Router# show interfaces
```

```
FastEthernet0/0 is up, line protocol is up
```

```
Hardware is Fast Ethernet
```

```
MTU 1500 bytes
```

```
BW 100000 Kbit
```

5 minute input rate 2000 bits/sec

5 minute output rate 1000 bits/sec

(k) Knowing the controller type (DTE or DCE)

Router# show controllers serial 0/0/0

Interface Serial0/0/0

Hardware is HD64570

DCE V.35 cable

The DCE device provides the clock signal on the serial link, while the DTE device receives the clock signal from the DCE.

(l) Configuring serial and Ethernet interfaces (IP address, mask, clock rate)

Ethernet Configuration:

Router# configure terminal

Router(config)# interface fastethernet0/0

Router(config-if)# ip address 192.168.1.1 255.255.255.0

Router(config-if)# no shutdown

Verification:

Router# show ip interface brief

FastEthernet0/0 192.168.1.1 YES manual up up

Serial Configuration:

Router(config)# interface serial0/0/0

Router(config-if)# ip address 10.0.0.1 255.0.0.0

Router(config-if)# clock rate 64000

Router(config-if)# no shutdown

Verification:

Router# show ip interface brief

Serial0/0/0 10.0.0.1 YES manual up up

EXPECTED OUTPUT

```
FastEthernet0/0 192.168.1.1 YES manual up up
```

```
Serial0/0/0 10.0.0.1 YES manual up up
```

Both interfaces show a status of "up/up", confirming that the IP addresses were assigned and the interfaces were enabled correctly.

MODEL VIVA QUESTIONS

1. What are the three basic modes of the Cisco router CLI, and how do you move between them?
2. What is the difference between the running-config and the startup-config?
3. Which command is used to save the running configuration permanently?
4. What information does the show ip route command provide, and what do the letters 'C' and 'S' mean in its output?
5. What is the difference between a DTE and a DCE device on a serial link?
6. Which command sets the clock rate on a serial interface, and on which end (DTE or DCE) is it configured?
7. What is the purpose of the no shutdown command on an interface?
8. How would you verify that an interface has been correctly configured and is operational?

RESULT

The basic Cisco router commands were successfully explored, including mode switching, viewing router and interface information, routing tables, and command history. Ethernet and serial interfaces were configured with IP addresses and a clock rate, and the configuration was verified to be up and operational using show commands.

EXP NO: 12**CONFIGURATION OF STATIC ROUTING****AIM:**

To set up static routing for a network consisting of four routers (2621, 2501A, 2501B, and 2501C), display the routing table on each router, and verify end-to-end connectivity across all networks using the ping command.

LEARNING OUTCOMES:

- Understand the concept of static routing and when it is used in place of dynamic routing.
- Learn the syntax and use of the ip route command to configure a static route.
- Understand the meaning of the destination network, subnet mask, and next-hop address in a static route entry.
- Learn to interpret the output of the show ip route command, including the codes C (connected) and S (static).
- Verify inter-network connectivity using the ping command and interpret its success rate.

DESCRIPTION:

Static routing is a method of configuring routes manually on a router, where the network administrator explicitly specifies the path that packets destined for a particular remote network must take. Each static route is configured using the command `ip route <destination-network> <subnet-mask> <next-hop-address>`, entered in global configuration mode. Unlike dynamic routing protocols, static routes do not change automatically in response to topology changes and must be updated manually if the network changes; however, they consume no bandwidth for route advertisements and are well suited to small, stable networks or stub networks with a single point of entry.

In the network used for this experiment, four routers - 2621, 2501A, 2501B, and 2501C - are connected in series through Ethernet and serial links, each network segment using a /24 mask. Because a router has direct knowledge only of the networks connected to its own interfaces (shown as "C" - connected routes - in the routing table), a static route must be added on every router for every remote network that is not directly attached to it, specifying the next-hop address of the neighbouring router that leads toward that destination. Once configured, the routing table on each router is

displayed using show ip route, and end-to-end reachability is confirmed by pinging addresses on the remote LANs from each router.

ALGORITHM

1. Access each router through the console and enter Privileged EXEC mode using enable.
2. Enter Global Configuration mode using the configure terminal.
3. On router 2621, add static routes for all remote networks (172.16.20.0, 172.16.30.0, and 172.16.40.0) with the next-hop address of router 2501A's connected interface.
4. On router 2501A, add static routes for the networks 172.16.30.0 and 172.16.40.0 with the next-hop address of router 2501B's connected interface.
5. On router 2501B, add a static route for the network 172.16.10.0 with the next-hop address of router 2501A's connected interface.
6. On router 2501C, add static routes for the networks 172.16.10.0, 172.16.20.0, and 172.16.30.0 with the next-hop address of router 2501B's connected interface.
7. Display the routing table on each router using show ip route and verify that all static routes appear with the code 'S'.
8. Verify connectivity from router 2621 to the LAN interface of router 2501B using ping.
9. Verify connectivity from router 2621 to router 2501C using ping.
10. Verify connectivity from router 2501C back to router 2621 using ping.
11. Confirm that all ping tests report a 100 percent success rate.

PROGRAM:

As this experiment involves configuring routers rather than writing a host program, the "program" consists of the following IOS configuration commands entered on each router.

Router 2621

Router>enable

Router#configure terminal

Router(config)# ip route 172.16.20.0 255.255.255.0 172.16.10.2

Router(config)# ip route 172.16.30.0 255.255.255.0 172.16.10.2

```
Router(config)# ip route 172.16.40.0 255.255.255.0 172.16.10.2
```

```
Router 2501A
```

```
Router>enable
```

```
Router#configure terminal
```

```
Router(config)# ip route 172.16.30.0 255.255.255.0 172.16.20.2
```

```
Router(config)# ip route 172.16.40.0 255.255.255.0 172.16.20.2
```

```
Router 2501B
```

```
Router>enable
```

```
Router#configure terminal
```

```
Router(config)# ip route 172.16.10.0 255.255.255.0 172.16.20.1
```

```
Router 2501C
```

```
Router>enable
```

```
Router#configure terminal
```

```
Router(config)# ip route 172.16.10.0 255.255.255.0 172.16.40.1
```

```
Router(config)# ip route 172.16.20.0 255.255.255.0 172.16.40.1
```

```
Router(config)# ip route 172.16.30.0 255.255.255.0 172.16.40.1
```

PROCEDURE:

1. Connect to each of the four routers (2621, 2501A, 2501B, 2501C) through the console port.
2. On each router, enter Privileged EXEC mode using enable, then Global Configuration mode using configure terminal.
3. Enter the static route commands listed in the PROGRAM section on the corresponding router, one ip route command for every remote network not directly connected to that router.
4. Exit configuration mode and display the routing table on each router using show ip route to confirm that the static routes have been added.
5. On router 2621, verify connectivity to the LAN of router 2501B by executing ping 172.16.30.1 and confirm a 100 percent success rate.
6. On router 2621, verify connectivity to router 2501C by executing ping 172.16.40.2 and confirm a 100 percent success rate.

7. On router 2501C, verify connectivity back to router 2621 by executing ping 172.16.10.1 and confirm a 100 percent success rate.
8. Repeat the ping tests between other router pairs as required to confirm full end-to-end reachability across the network.

EXPECTED OUTPUT

Routing table on Router 2621:

Router# show ip route

C 172.16.10.0/24 is directly connected, FastEthernet0/0

S 172.16.20.0/24 [1/0] via 172.16.10.2

S 172.16.30.0/24 [1/0] via 172.16.10.2

S 172.16.40.0/24 [1/0] via 172.16.10.2

Connectivity verification:

Router# ping 172.16.30.1

Success rate is 100 percent (5/5)

Router# ping 172.16.40.2

Success rate is 100 percent (5/5)

Router# ping 172.16.10.1

Success rate is 100 percent (5/5)

MODEL VIVA QUESTIONS:

1. What is static routing, and how does it differ from dynamic routing?
2. What is the syntax of the ip route command, and what does each parameter represent?
3. What do the codes 'C' and 'S' represent in the output of show ip route?
4. What is the administrative distance of a static route, and why does it matter?
5. What are the advantages and disadvantages of static routing compared to a dynamic routing protocol such as RIP or OSPF?
6. What would happen to connectivity if a link in the network failed, when using static routing?

RESULT

Static routes were successfully configured on all four routers. The routing tables displayed the configured static routes, and connectivity between all networks was verified successfully using the ping command. Hence, static routing was implemented and tested successfully.

EXP NO: 13**IMPLEMENTATION OF RIPv2 ROUTING****AIM**

To configure RIPv2 (Routing Information Protocol version 2) dynamic routing on the routers in a given network topology, display the resulting routing tables, and verify connectivity across all networks using the ping command.

LEARNING OUTCOMES

- Understand the working principle of RIP as a distance-vector routing protocol.
- Learn the differences between RIPv1 and RIPv2, particularly regarding subnet masks and auto-summarization.
- Learn to configure RIPv2 using the router rip, version 2, no auto-summary, and network commands.
- Learn to verify a routing protocol's configuration using show ip protocols.
- Learn to interpret dynamically learned routes (code 'R') in the output of show ip route, including the metric (hop count) and administrative distance of RIP.
- Verify connectivity across a dynamically routed network using the ping command.

DESCRIPTION

The Routing Information Protocol (RIP) is a distance-vector interior gateway protocol that determines the best path to a destination network based solely on hop count, with a maximum of 15 hops considered reachable. Each router running RIP periodically broadcasts (or multicasts, in RIPv2) its entire routing table to its directly connected neighbours, allowing routers to learn about networks that are not directly attached to them without requiring manual configuration, unlike static routing.

RIPv2, used in this experiment, is an enhancement over the original RIPv1 that supports Variable Length Subnet Masking (VLSM) by carrying subnet mask information along with each route advertisement, and it sends updates as multicasts to 224.0.0.9 rather than broadcasts. RIPv2 is configured on a router using router rip to enter router configuration mode, version 2 to enable RIPv2 behaviour, no auto-summary to disable automatic summarization at classful network boundaries (important when subnets are used), and one or more network statements identifying the directly connected networks that should participate in RIP. Every router in the topology advertises its directly

connected networks to its neighbours, and after the protocol converges, each router's routing table contains both its own connected networks (marked 'C') and routes learned dynamically via RIP (marked 'R'), each carrying an administrative distance of 120 and a metric equal to the hop count to that network.

ALGORITHM

1. Access each router through the console and enter Privileged EXEC mode using enable, then Global Configuration mode using configure terminal.
2. On each router, enter router configuration mode for RIP using router rip.
3. Enable RIPv2 on each router using the command version 2.
4. Disable automatic route summarization on each router using no auto-summary.
5. On router 2621, advertise the directly connected network 172.16.10.0 using the network command.
6. On router 2501A, advertise its two directly connected networks, 172.16.10.0 and 172.16.20.0.
7. On router 2501B, advertise its three directly connected networks, 172.16.20.0, 172.16.30.0, and 172.16.40.0.
8. On router 2501C, advertise its directly connected network, 172.16.40.0.
9. Verify that RIP is running correctly on each router using show ip protocols.
10. Display the routing table on each router using show ip route and confirm that remote networks appear as RIP-learned routes (code 'R').
11. Verify connectivity from router 2621 to the LAN of router 2501B using ping.
12. Verify connectivity from router 2621 to router 2501C using ping.
13. Verify connectivity from router 2501C back to router 2621 using ping.
14. Confirm that all ping tests report a 100 percent success rate.

PROGRAM

As this experiment involves configuring routers rather than writing a host program, the "program" consists of the following IOS configuration commands entered on each router.

Router 2621

```
Router>enable

Router#configure terminal

Router(config)# router rip

Router(config-router)# version 2

Router(config-router)# no auto-summary

Router(config-router)# network 172.16.10.0

Router 2501A

Router>enable

Router#configure terminal

Router(config)# router rip

Router(config-router)# version 2

Router(config-router)# no auto-summary

Router(config-router)# network 172.16.10.0

Router(config-router)# network 172.16.20.0

Router 2501B

Router>enable

Router#configure terminal

Router(config)# router rip

Router(config-router)# version 2

Router(config-router)# no auto-summary

Router(config-router)# network 172.16.20.0

Router(config-router)# network 172.16.30.0

Router(config-router)# network 172.16.40.0

Router 2501C

Router>enable

Router#configure terminal

Router(config)# router rip
```

```
Router(config-router)# version 2  
Router(config-router)# no auto-summary  
Router(config-router)# network 172.16.40.0
```

PROCEDURE

1. Connect to each of the four routers (2621, 2501A, 2501B, 2501C) through the console port.
2. On each router, enter Privileged EXEC mode using enable, then Global Configuration mode using configure terminal.
3. Enter router configuration mode using router rip, then enable RIPv2 and disable auto-summarization using version 2 and no auto-summary.
4. Enter a network statement for every directly connected network on that router, as listed in the PROGRAM section.
5. Exit configuration mode and, on any router, verify the RIP configuration using show ip protocols; confirm that the correct networks are listed under "Routing for Networks".
6. Allow a short interval for RIP updates to be exchanged and the network to converge.
7. Display the routing table on each router using show ip route and confirm that remote networks are learned dynamically, marked with the code 'R' and an administrative distance of 120.
8. On router 2621, verify connectivity to the LAN of router 2501B by executing ping 172.16.30.1 and confirm a 100 percent success rate.
9. On router 2621, verify connectivity to router 2501C by executing ping 172.16.40.2 and confirm a 100 percent success rate.
10. On router 2501C, verify connectivity back to router 2621 by executing ping 172.16.10.1 and confirm a 100 percent success rate.

EXPECTED OUTPUT

RIP configuration verification:

Router# show ip protocols

Routing Protocol is "rip"

Sending updates every 30 seconds

Invalid after 180 seconds

Hold down 180 seconds

Flush after 240 seconds

Routing for Networks:

172.16.10.0

172.16.20.0

Routing table on Router 2621:

Router# show ip route

C 172.16.10.0/24 is directly connected, FastEthernet0/0

R 172.16.20.0/24 [120/1] via 172.16.10.2

R 172.16.30.0/24 [120/2] via 172.16.10.2

R 172.16.40.0/24 [120/3] via 172.16.10.2

Connectivity verification:

Router# ping 172.16.30.1

Success rate is 100 percent (5/5)

Router# ping 172.16.40.2

Success rate is 100 percent (5/5)

Router# ping 172.16.10.1

Success rate is 100 percent (5/5)

MODEL VIVA QUESTIONS

1. What type of routing protocol is RIP, and on what metric does it base its routing decisions?
2. What is the maximum hop count supported by RIP, and what does a hop count of 16 indicate?
3. What are the key differences between RIPv1 and RIPv2?
4. Why is the no auto-summary command needed when configuring RIPv2 in a subnetted network?
5. What is the administrative distance of RIP, and how does it compare with that of a static route and OSPF?

RESULT

RIPv2 routing was successfully configured on all four routers. The routing tables contained dynamically learned RIP routes for every remote network, and connectivity between all networks was verified successfully using the ping command. Hence, the implementation of RIPv2 routing was successful.

EXP NO: 14**IMPLEMENTATION OF OSPF ROUTING****AIM:**

To configure OSPF (Open Shortest Path First) routing on the routers in a given network topology, display the resulting routing tables, and verify connectivity across all networks using the ping command.

LEARNING OUTCOMES:

- Understand the working principle of OSPF as a link-state routing protocol. Learn to configure OSPF using the router ospf, and network <address> <wildcard-mask> area <area-id> commands.
- Understand the role of the OSPF process ID, area ID, and wildcard mask in the configuration.
- Learn to verify OSPF neighbour relationships using show ip ospf neighbor.
- Learn to interpret dynamically learned OSPF routes (code 'O') and the OSPF administrative distance in show ip route.
- Understand the purpose of the OSPF link-state database, viewed using show ip ospf database.
- Verify connectivity across an OSPF-routed network using the ping command.

DESCRIPTION:

OSPF (Open Shortest Path First) is a link-state interior gateway protocol that, unlike the distance-vector protocol RIP, requires every router to build a complete map (the link-state database) of the network topology within its area and independently compute the shortest path to every destination using Dijkstra's shortest-path-first algorithm. Because routing decisions are based on a configurable cost (derived from interface bandwidth by default) rather than a simple hop count, OSPF scales far better than RIP and is widely used in medium and large enterprise networks.

OSPF is configured on a router using router ospf <process-id> to start the OSPF process (the process ID is locally significant and need not match between routers), followed by one or more network <network-address> <wildcard-mask> area <area-id> statements that identify which directly connected interfaces participate in OSPF and to which area they belong. In this experiment all routers belong to a single backbone area, area 0. Once configured, neighbouring routers exchange Hello packets to discover each other and form an adjacency, exchange link-state advertisements (LSAs) to

build an identical link-state database, and finally compute their routing tables. Routes learned via OSPF appear in the routing table with the code 'O', an administrative distance of 110, and a cost value that reflects the cumulative path cost to that destination.

ALGORITHM:

1. Access each router through the console and enter Privileged EXEC mode using enable, then Global Configuration mode using configure terminal.
2. On each router, start the OSPF process using router ospf 1.
3. On router 2621, advertise the directly connected network 172.16.10.0 in area 0 using an appropriate network statement.
4. On router 2501A, advertise its two directly connected networks, 172.16.10.0 and 172.16.20.0, in area 0.
5. On router 2501B, advertise its three directly connected networks, 172.16.20.0, 172.16.30.0, and 172.16.40.0, in area 0.
6. On router 2501C, advertise its directly connected network, 172.16.40.0, in area 0.
7. Verify that OSPF neighbour adjacencies have been formed using show ip ospf neighbor.
8. Display the routing table on each router using show ip route and confirm that remote networks appear as OSPF-learned routes (code 'O').
9. Display the OSPF link-state database on a router using show ip ospf database.
10. Verify connectivity from router 2621 to the LAN of router 2501B using ping.
11. Verify connectivity from router 2621 to router 2501C using ping.
12. Verify connectivity from router 2501C back to router 2621 using ping.
13. Confirm that all ping tests report a 100 percent success rate.

PROGRAM

As this experiment involves configuring routers rather than writing a host program, the "program" consists of the following IOS configuration commands entered on each router.

Router 2621

Router>enable

```
Router#configure terminal
```

```
Router(config)# router ospf 1
```

```
Router(config-router)# network 172.16.10.0 0.0.0.255 area 0
```

Router 2501A

```
Router>enable
```

```
Router#configure terminal
```

```
Router(config)# router ospf 1
```

```
Router(config-router)# network 172.16.10.0 0.0.0.255 area 0
```

```
Router(config-router)# network 172.16.20.0 0.0.0.255 area 0
```

Router 2501B

```
Router>enable
```

```
Router#configure terminal
```

```
Router(config)# router ospf 1
```

```
Router(config-router)# network 172.16.20.0 0.0.0.255 area 0
```

```
Router(config-router)# network 172.16.30.0 0.0.0.255 area 0
```

```
Router(config-router)# network 172.16.40.0 0.0.0.255 area 0
```

Router 2501C

```
Router>enable
```

```
Router#configure terminal
```

```
Router(config)# router ospf 1
```

```
Router(config-router)# network 172.16.40.0 0.0.0.255 area 0
```

PROCEDURE

1. Connect to each of the four routers (2621, 2501A, 2501B, 2501C) through the console port.
2. On each router, enter Privileged EXEC mode using enable, then Global Configuration mode using configure terminal.
3. Start the OSPF process using router ospf 1, and enter a network <address> <wildcard-mask> area 0 statement for every directly connected network on that router, as listed in the PROGRAM section.

4. Exit configuration mode and allow a short interval for OSPF Hello packets to be exchanged and adjacencies to form.
5. On any router, verify that neighbour adjacencies have reached the FULL state using `show ip ospf neighbor`.
6. Display the routing table on each router using `show ip route` and confirm that remote networks are learned dynamically, marked with the code 'O' and an administrative distance of 110.
7. Optionally, display the link-state database using `show ip ospf database` to view the LSAs exchanged among the OSPF routers.
8. On router 2621, verify connectivity to the LAN of router 2501B by executing `ping 172.16.30.1` and confirm a 100 percent success rate.
9. On router 2621, verify connectivity to router 2501C by executing `ping 172.16.40.2` and confirm a 100 percent success rate.
10. On router 2501C, verify connectivity back to router 2621 by executing `ping 172.16.10.1` and confirm a 100 percent success rate.

EXPECTED OUTPUT

OSPF neighbour verification:

Router# `show ip ospf neighbor`

Neighbor ID	Pri	State	Dead Time	Address
-------------	-----	-------	-----------	---------

172.16.20.2	1	FULL/	- 00:00:38	172.16.20.2
-------------	---	-------	------------	-------------

Routing table on Router 2621:

Router# `show ip route`

C 172.16.10.0/24 is directly connected, FastEthernet0/0

O 172.16.20.0/24 [110/65] via 172.16.10.2

O 172.16.30.0/24 [110/129] via 172.16.10.2

O 172.16.40.0/24 [110/193] via 172.16.10.2

Connectivity verification:

Router# ping 172.16.30.1

Success rate is 100 percent (5/5)

Router# ping 172.16.40.2

Success rate is 100 percent (5/5)

Router# ping 172.16.10.1

Success rate is 100 percent (5/5)

MODEL VIVA QUESTIONS

1. What type of routing protocol is OSPF, and how does it differ from a distance-vector protocol such as RIP?
2. What algorithm does OSPF use internally to compute the shortest path to each destination?
3. What is the significance of the process ID in the router ospf <process-id> command?
4. What does the wildcard mask represent in an OSPF network statement, and how is it different from a subnet mask?
5. What is an OSPF area, and why is area 0 referred to as the backbone area?
6. What is the administrative distance of OSPF, and how does it compare with RIP and static routes?
7. What states does an OSPF neighbour relationship pass through before reaching FULL?
8. What is stored in the OSPF link-state database, and which command is used to view it?

RESULT

OSPF routing was successfully configured on all four routers. The routing tables displayed OSPF-learned routes for every remote network, and connectivity between all networks was verified successfully using the ping command. Hence, the implementation of OSPF routing was successful.

EXP NO: 15**CONFIGURATION OF AN EXTENDED ACCESS CONTROL LIST (ACL)****AIM**

To configure an Extended Access Control List (ACL) on a campus network so that only Host_B is permitted to communicate with the network 172.16.10.0/24, while access from Lab_B and Lab_C - which reside on the same subnet as Host_B - is denied, and to verify the configuration using ping tests.

LEARNING OUTCOMES

- Understand the purpose of Access Control Lists (ACLs) in restricting network traffic.
- Learn the difference between a Standard ACL and an Extended ACL, and when an Extended ACL is required.
- Learn the syntax of the access-list command for permitting or denying traffic based on source and destination address.
- Understand the significance of the implicit "deny any" at the end of every ACL, and why an explicit permit ip any any statement is needed.
- Learn to apply an ACL to a router interface in the inbound direction using the ip access-group command.
- Learn to verify an ACL's configuration and match statistics using show access-lists.
- Understand why the order of statements within an ACL is significant.

DESCRIPTION

An Access Control List (ACL) is an ordered list of permit or deny statements that a router applies to packets, allowing the network administrator to control which traffic is allowed to pass through an interface. A Standard ACL can filter traffic only on the basis of the source IP address, whereas an Extended ACL can filter on source address, destination address, protocol (such as TCP, UDP, or ICMP), and even port number, making it suitable for the fine-grained requirement in this experiment: permitting one specific host, while denying its neighbouring hosts on the same subnet.

In the given topology, Host_A resides on the network 172.16.10.0/24, while Host_B, Lab_B, and Lab_C are all connected to the network 172.16.30.0/24. Since Host_B, Lab_B, and Lab_C share the same subnet, a Standard ACL - which can only match on source network - cannot distinguish between

them; an Extended ACL is required to permit traffic specifically from Host_B's individual address while denying traffic from the rest of the 172.16.30.0/24 subnet. Every ACL ends with an implicit "deny any any", so a final permit ip any any statement is added to ensure that unrelated traffic elsewhere in the network is not accidentally blocked. The ACL is then applied inbound on the router interface connected to the 172.16.30.0/24 network, so that it is evaluated as traffic enters the router from that subnet.

ALGORITHM

1. Access the router through the console and enter Privileged EXEC mode using enable, then Global Configuration mode using configure terminal.
2. Create an extended numbered ACL (access-list 101) with a first statement that explicitly permits IP traffic from Host_B (172.16.30.10) to the destination network 172.16.10.0/24.
3. Add a second statement to the same ACL that denies IP traffic from the entire source network 172.16.30.0/24 to the destination network 172.16.10.0/24.
4. Add a final statement, permit ip any any, so that all other traffic not related to this restriction continues to be forwarded normally.
5. Enter interface configuration mode for the interface connected to the 172.16.30.0/24 network (f0/0).
6. Apply the ACL to that interface in the inbound direction using ip access-group 101 in.
7. Verify the configured ACL using show access-lists.
8. From Host_B, ping Host_A (172.16.10.10) and confirm that the ping succeeds.
9. From Lab_B, ping Host_A and confirm that the ping is blocked (times out).
10. From Lab_C, ping Host_A and confirm that the ping is blocked (times out).

PROGRAM

As this experiment involves configuring a router's access list rather than writing a host program, the "program" consists of the following IOS configuration commands.

```
Router(config)# access-list 101 permit ip host 172.16.30.10 172.16.10.0 0.0.0.255
```

```
Router(config)# access-list 101 deny ip 172.16.30.0 0.0.0.255 172.16.10.0 0.0.0.255
```

```
Router(config)# access-list 101 permit ip any any
```

```
Router(config)# interface f0/0
```

```
Router(config-if)# ip access-group 101 in
```

PROCEDURE

1. Note the IP addresses involved: Host_A = 172.16.10.10, Host_B = 172.16.30.10, Lab_B = 172.16.30.11, Lab_C = 172.16.30.12.
2. Connect to the router through the console and enter Privileged EXEC mode using enable, then Global Configuration mode using configure terminal.
3. Enter the three access-list 101 statements listed in the PROGRAM section, in the exact order shown, since ACL statements are evaluated sequentially.
4. Enter interface configuration mode for interface f0/0 (the interface connected to the 172.16.30.0/24 network) and apply the ACL inbound using ip access-group 101 in.
5. Exit configuration mode and confirm the ACL has been entered correctly using show access-lists.
6. From Host_B's command prompt, execute ping 172.16.10.10 and verify that replies are received with a 100 percent success rate.
7. From Lab_B's command prompt, execute ping 172.16.10.10 and verify that the request times out with a 0 percent success rate.
8. From Lab_C's command prompt, execute ping 172.16.10.10 and verify that the request times out with a 0 percent success rate.

EXPECTED OUTPUT

ACL configuration:

```
Router# show access-lists
```

```
Extended IP access list 101
```

```
10 permit ip host 172.16.30.10 172.16.10.0 0.0.0.255
```

```
20 deny ip 172.16.30.0 0.0.0.255 172.16.10.0 0.0.0.255
```

```
30 permit ip any any
```

(a) Ping Host_A from Host_B:

```
Host_B> ping 172.16.10.10
```

```
Reply from 172.16.10.10
```

```
Reply from 172.16.10.10
```

```
Reply from 172.16.10.10
```

```
Reply from 172.16.10.10
```

```
Success rate is 100%
```

(b) Ping Host_A from Lab_B and Lab_C:

```
Lab_B> ping 172.16.10.10
```

```
Request timed out.
```

```
Request timed out.
```

```
Request timed out.
```

```
Request timed out.
```

```
Success rate is 0%
```

```
Lab_C> ping 172.16.10.10
```

```
Request timed out.
```

```
Request timed out.
```

```
Request timed out.
```

```
Request timed out.
```

```
Success rate is 0%
```

MODEL VIVA QUESTIONS

1. What is the difference between a Standard ACL and an Extended ACL?
2. Why can a Standard ACL not be used to solve this particular requirement?

3. What is the implicit rule at the end of every ACL, and why is a permit ip any any statement added here?
4. Why is the ACL applied in the inbound direction on interface f0/0 rather than outbound?
5. What would happen if the permit statement for Host_B were placed after the deny statement for the 172.16.30.0/24 network?
6. What is the range of ACL numbers used for extended numbered IP ACLs?

RESULT

An Extended Access Control List (ACL) was successfully configured to allow only Host_B to communicate with the 172.16.10.0/24 network. Connectivity tests confirmed that Host_B could successfully ping Host_A, while Lab_B and Lab_C were denied access. Thus, the ACL configuration was verified successfully.

EXP NO: 16**SUBNETTING AND EXTENDED ACL CONFIGURATION FOR SELECTIVE SERVICE
BLOCKING****AIM**

To design a subnetting scheme for a given Class B network so as to satisfy a specified number of subnets, to locate a specific host address within a given subnet, and to configure an extended Access Control List (ACL) on a router that blocks only a single service (HTTP) between two subnets while permitting all other IP traffic between them.

LEARNING OUTCOMES

1. Determine the number of host bits that must be borrowed to create a required number of subnets from a given major network.
2. Calculate the new subnet mask, block size, and the network address of any specific subnet within the major network.
3. Identify a particular host address within a subnet using host offsets from the subnet's network address.
4. Differentiate between standard and extended ACLs and identify the situations in which each is appropriate.
5. Configure an extended ACL that filters traffic based on source network, destination host, protocol, and port number.
6. Apply an ACL to the correct router interface and in the correct direction, and verify that only the intended service is blocked while all other services continue to function.

DESCRIPTION

Subnetting is the process of dividing a large network into smaller logical networks (subnets) by borrowing bits from the host portion of an IP address to extend the network portion. If N subnets are required, the number of bits (n) to be borrowed must satisfy $2^n \geq N$. Each borrowed bit doubles the number of subnets and halves the number of hosts available per subnet. Once the number of borrowed bits is known, the new subnet mask is obtained by setting the corresponding number of additional bits to 1 in the default mask, and the block size (the size of each subnet) is calculated as 256 minus the value of the octet in which the mask changes. Successive subnets are then obtained by repeatedly

adding the block size to the network address, and individual host addresses within a subnet are obtained by adding a host offset to that subnet's network address.

An Access Control List (ACL) is a sequential set of permit or deny statements that a router uses to filter traffic. Standard ACLs (numbered 1–99) can filter only on the source IP address, whereas Extended ACLs (numbered 100–199) can filter on source address, destination address, protocol (such as TCP or UDP), and port number (such as port 80 for HTTP), allowing much finer control. Because an ACL is evaluated top-to-bottom and stops at the first match, a specific deny statement must be placed before a more general permit statement; otherwise the general statement would match first and the intended restriction would never be enforced. Every ACL also ends with an implicit "deny any any", so an explicit "permit ip any any" statement must be added at the end whenever other, unrelated traffic is meant to be allowed through. The wildcard mask used in an ACL statement is the inverse of the subnet mask (obtained by subtracting each octet of the subnet mask from 255) and specifies which address bits must match exactly and which may vary.

ALGORITHM

1. Start.
2. Read the given major network address and the required number of subnets, N .
3. Find the smallest integer n such that $2^n \geq N$; this is the number of host bits to be borrowed.
4. Compute the new subnet mask by setting n additional bits to 1 in the default mask, and compute the block size as 256 minus the value of the octet in which the borrowing occurs.
5. Generate the list of subnets by repeatedly adding the block size to the network address, starting from the given major network address.
6. Identify the subnet containing the source host (CCF) and the subnet containing the destination host (server) from the generated list.
7. Compute the wildcard mask by subtracting each octet of the subnet mask from 255.
8. Compute the exact IP address of the destination server by adding the given host offset to the network address of its subnet.
9. Write a deny statement in the extended ACL that matches the source subnet (with its wildcard mask), the destination host, the TCP protocol, and the destination port 80 (HTTP).
10. Write a permit ip any any statement immediately after the deny statement so that all other traffic is allowed.
11. Apply the ACL to the appropriate router interface in the inbound direction using the ip access-group command.
12. Test HTTP access from the source subnet to the server and confirm that it is denied.

13. Test other services (ping, FTP, SSH) from the source subnet to the server and confirm that they are permitted.
14. Stop.

PROCEDURE

1. Note the given major network address and the number of subnets required.
2. Find the smallest value of n such that 2^n is greater than or equal to the number of required subnets, and borrow n bits from the host portion of the address.
3. Compute the new subnet mask by setting the borrowed bits to 1, and compute the block size in the octet where the borrowing takes place.
4. List the successive subnets by repeatedly adding the block size to the network address, and identify the network address of the subnet in which the source host (CCF) resides and the subnet in which the destination host (server) resides.
5. Compute the wildcard mask corresponding to the subnet mask by subtracting each octet of the mask from 255.
6. Locate the specific host address of the server by adding the given host offset to the network address of its subnet.
7. On the router, write an extended ACL: first, a deny statement matching the source subnet, the destination host, the TCP protocol, and the HTTP port (80); second, a permit ip any any statement so that every other type of traffic is still allowed.
8. Apply the ACL to the router interface connected to the source subnet in the inbound direction using the ip access-group command.
9. Verify the configuration using show access-lists, and test connectivity for HTTP as well as for other services such as ping, FTP, or SSH.

Subnetting the given network 140.80.0.0 for 20 subnets

Number of subnets required = 20

Smallest n such that $2^n \geq 20 \rightarrow n = 5$ ($2^5 = 32$)

Bits borrowed = 5 New mask = /16 + 5 = /21

Subnet mask = 255.255.248.0

Block size in the 3rd octet = $256 - 248 = 8$

Resulting subnets (block size 8 in the 3rd octet)

Subnet 1 : 140.80.0.0/21

Subnet 2 : 140.80.8.0/21

Subnet 3 : 140.80.16.0/21

Subnet 4 : 140.80.24.0/21 → CCF network

Subnet 5 : 140.80.32.0/21...

Subnet 16 : 140.80.120.0/21 → Server network

Wildcard mask and server host address

Subnet mask 255.255.248.0 → Wildcard mask 0.0.7.255

16th subnet 140.80.120.0/21, 7th host → Server IP = 140.80.120.7

Extended ACL configuration

```
Router(config)# access-list 101 deny tcp 140.80.24.0 0.0.7.255 host 140.80.120.7 eq 80
```

```
Router(config)# access-list 101 permit ip any any
```

```
Router(config)# interface GigabitEthernet0/0
```

```
Router(config-if)# ip access-group 101 in
```

EXPECTED OUTPUT

show access-lists on the router:

Extended IP access list 101

10 deny tcp 140.80.24.0 0.0.7.255 host 140.80.120.7 eq www

20 permit ip any any

Connectivity tests from a host on the CCF subnet (140.80.24.0/21):

http://140.80.120.7 → Access Denied (HTTP blocked)

ping 140.80.120.7 → Reply from 140.80.120.7 (allowed)

ftp 140.80.120.7 / ssh 140.80.120.7 → Connection Successful (allowed)

MODEL VIVA QUESTIONS:

1. What is the difference between a standard ACL and an extended ACL?
2. How many subnets and usable hosts per subnet result from borrowing 5 bits from a /16 network?
3. How is the wildcard mask derived from a subnet mask, and why does an ACL use a wildcard mask instead of a subnet mask?
4. Why must the "permit ip any any" statement be placed after the deny statement and not before it?
5. What happens to traffic that does not match any statement in an ACL?
6. In which direction (in/out) and on which interface would you apply this ACL, and why does that placement matter?

RESULT

The network 140.80.0.0 was successfully subnetted into 20 subnets using a /21 mask (255.255.248.0), and the network addresses of the CCF subnet (4th subnet) and the server subnet (16th subnet) were identified. An extended ACL was configured and applied to block only HTTP (port 80) traffic from the CCF subnet to the hackathon server while permitting all other services. Connectivity tests confirmed that web access to the server was denied from CCF while ping, FTP, and SSH access remained unaffected. Hence, the subnetting scheme and the selective extended ACL were implemented and verified successfully.

EXP NO: 17**INTERCONNECTION OF IPv6 SUBNETS USING RIPng****AIM:**

To assign global unicast IPv6 addresses to the router and host interfaces of a given multi-router topology, to interconnect the different IPv6 subnets by configuring RIPng (RIP next generation) as the dynamic routing protocol, and to verify end-to-end connectivity across all subnets using ping.

LEARNING OUTCOMES:

1. Assign global unicast IPv6 addresses to router interfaces and end-host (PC) interfaces from a given addressing scheme.
2. Enable IPv6 routing capability on a Cisco router using the `ipv6 unicast-routing` command
3. Configure and enable a RIPng process on the interfaces that must participate in dynamic IPv6 routing.
4. Examine and interpret directly connected and RIPng-learned routes in the IPv6 routing table.
5. Verify end-to-end IPv6 connectivity across multiple subnets using the ping command with IPv6 addresses.

DESCRIPTION:

IPv6 uses 128-bit addresses, commonly written as eight groups of hexadecimal digits, and supports several address types including global unicast addresses (used for routable end-to-end communication) and link-local addresses (automatically assigned from the `FE80::/10` range and used for communication and next-hop advertisement on a single link). Unlike IPv4, IPv6 routing must be explicitly enabled on a Cisco router using the global command `ipv6 unicast-routing` before any IPv6 routing protocol can operate.

RIPng (RIP next generation) is the IPv6 counterpart of RIPv2. It is a distance-vector protocol that uses hop count as its metric, supports a maximum of 15 hops, uses UDP port 521, and multicasts its updates to the reserved multicast address `FF02::9` (all RIP routers) using the link-local address of the sending interface as the next hop. RIPng is enabled per interface (rather than through a network

statement, as in IPv4 RIP) using the command `ipv6 rip <process-name> enable` under each participating interface, after first creating the process implicitly through this command.

ALGORITHM:

1. Start.
2. Read the given topology and note the IPv6 address to be assigned to every router interface and every host (PC) interface.
3. For each router, enter global configuration mode and enable IPv6 routing capability using the command `ipv6 unicast-routing`.
4. For each interface on the router that must participate in routing, enter interface configuration mode and assign the given IPv6 address and prefix length using `ipv6 address <address>/<prefix-length>`.
5. On the same interface, enable RIPng using a common process name across the whole topology, using the command `ipv6 rip <process-name> enable`.
6. Activate the interface using `no shutdown`, and repeat steps 4 to 6 for every participating interface on every router.
7. Assign the given IPv6 address and default gateway to each PC in the topology.
8. On each router, verify that the RIPng process is running and bound to the correct interfaces using `show ipv6 protocols`.
9. Display the IPv6 routing table of each router using `show ipv6 route` and verify that directly connected networks are marked C and RIPng-learned networks are marked R with the correct next-hop link-local address.
10. From a host in one subnet, ping a host in every other subnet and verify that all pings succeed with a 100% success rate.
11. Stop.

PROCEDURE:

1. Note the given IPv6 addressing scheme for every router interface and host (PC) interface, and identify the subnets to be interconnected.
2. On each router, enter global configuration mode and enable IPv6 routing using `ipv6 unicast-routing`.

3. On each router, configure the IPv6 address on every participating interface using the `ipv6 address <address>/<prefix-length>` command, and enable RIPng on that interface using `ipv6 rip <process-name> enable`, then bring up the interface with `no shutdown`.
4. Use the same RIPng process name (for example, RIPNG) on every router and every interface so that they form a single RIPng routing domain.
5. Configure the IPv6 address and default gateway on each PC based on the given addressing scheme.
6. Verify the RIPng process on each router using `show ipv6 protocols`.
7. Display the IPv6 routing table on each router using `show ipv6 route`, and confirm that directly connected subnets appear with the code C and RIPng-learned subnets appear with the code R.
8. Verify end-to-end reachability by pinging hosts in different subnets from PC-A, and confirm a 100% success rate.

IPv6 addressing scheme

PC-A NIC 2001:DB8:1111:1::1/64

R1 G0/0 2001:DB8:1111:1::1/64

R1 S0/0/0 2001:DB8:1111:4::1/64

R1 G0/1/0 2001:DB8:1111:3::1/64

R2 S0/0/1 2001:DB8:1111:4::2/64

R2 G0/0 2001:DB8:1111:2::2/64

R3 G0/0/0 2001:DB8:1111:3::3/64

R3 G0/0 2001:DB8:1111:5::3/64

R1 configuration

```
Router> enable
```

```
Router# configure terminal
```

```
Router(config)# ipv6 unicast-routing
```

```
Router(config)# interface g0/0
```

```
Router(config-if)# ipv6 address 2001:DB8:1111:1::1/64
```

```
Router(config-if)# ipv6 rip RIPNG enable
```

```
Router(config-if)# no shutdown
```

```
Router(config)# interface s0/0/0
```

```
Router(config-if)# ipv6 address 2001:DB8:1111:4::1/64
```

```
Router(config-if)# ipv6 rip RIPNG enable
```

```
Router(config-if)# no shutdown
```

```
Router(config)# interface g0/1/0
```

```
Router(config-if)# ipv6 address 2001:DB8:1111:3::1/64
```

```
Router(config-if)# ipv6 rip RIPNG enable
```

```
Router(config-if)# no shutdown
```

R2 and R3 configuration

```
R2(config)# ipv6 unicast-routing
```

```
R2(config)# interface s0/0/1
```

```
R2(config-if)# ipv6 address 2001:DB8:1111:4::2/64
```

```
R2(config-if)# ipv6 rip RIPNG enable
```

```
R2(config-if)# no shutdown
```

```
R2(config)# interface g0/0
```

```
R2(config-if)# ipv6 address 2001:DB8:1111:2::2/64
```

```
R2(config-if)# ipv6 rip RIPNG enable
```

```
R2(config-if)# no shutdown
```

```
R3(config)# ipv6 unicast-routing
```

```
R3(config)# interface g0/0/0

R3(config-if)# ipv6 address 2001:DB8:1111:3::3/64

R3(config-if)# ipv6 rip RIPNG enable

R3(config-if)# no shutdown

R3(config)# interface g0/0

R3(config-if)# ipv6 address 2001:DB8:1111:5::3/64

R3(config-if)# ipv6 rip RIPNG enable

R3(config-if)# no shutdown
```

PC configuration

PC-A : IPv6 2001:DB8:1111:1::11/64 Gateway 2001:DB8:1111:1::1

PC-B : IPv6 2001:DB8:1111:2::10/64 Gateway 2001:DB8:1111:2::2

PC-C : IPv6 2001:DB8:1111:5::10/64 Gateway 2001:DB8:1111:5::3

EXPECTED OUTPUT:

show ipv6 route on R1:

C 2001:DB8:1111:1::/64

C 2001:DB8:1111:3::/64

C 2001:DB8:1111:4::/64

R 2001:DB8:1111:2::/64 via FE80::2, Serial0/0/0

R 2001:DB8:1111:5::/64 via FE80::3, GigabitEthernet0/1/0

Connectivity verification:

PC-A> ping 2001:DB8:1111:2::10 → Reply received, success rate = 100%

PC-A> ping 2001:DB8:1111:5::10 → Reply received, success rate = 100%

MODEL VIVA QUESTIONS:

1. What is the purpose of the ipv6 unicast-routing command, and what happens if it is not configured?
2. How Does RIPng differ from RIPv2 in terms of configuration and operation?
3. Which multicast address does RIPng use to advertise routing updates, and What is its significance?
4. What is a link-local address, and what role does it play as the next-hop address in RIPng routes?
5. What is the maximum hop count supported by RIPng, and what does a route with a metric of 16 indicated?
6. What do the codes C and R represent in the output of show ipv6 route?

RESULT:

IPv6 addresses were successfully configured on all router and host interfaces as per the given addressing scheme. RIPng was enabled on all participating router interfaces using a common process name, allowing the routers to dynamically exchange IPv6 routing information. The routing tables on all routers were populated with the correct directly connected and RIPng-learned routes, and end-to-end connectivity between PC-A, PC-B, and PC-C across all subnets was verified successfully using ping. Hence, the different IPv6 subnets were successfully interconnected using RIPng.